NEA - Group Creator for School Projects

Amelie Machin

Contents

1 Analysis **2**
1.1 Problem statement . . . 2
1.2 Research . . . 2
1.2.1 Sources of inspiration . . . 2
1.2.2 Algorithm . . . 3
1.2.3 Inputting data . . . 4
1.2.4 Accounts . . . 4
1.2.5 Internal data storage . . . 5
1.2.6 External data storage . . . 5
1.2.7 GUI library . . . 5
1.3 Dialogue . . . 6
1.3.1 Initial interview . . . 6
1.3.2 Follow-up questions . . . 7
1.4 Objectives . . . 8
1.5 Modelling . . . 14

2 Design **17**
2.1 GUI . . . 17
2.2 Databases . . . 24
2.3 Inputs and outputs . . . 24
2.4 Algorithm . . . 26
2.4.1 Pseudocode . . . 26
2.4.2 Fitness algorithm . . . 28
2.4.3 Diagrams . . . 28

3 Technical solution **32**
3.1 main.py . . . 33
3.2 algoClasses.py . . . 57
3.3 geneticAlgo.py . . . 60
3.4 CreateTables.sql . . . 65

4 Testing **67**
4.1 Video . . . 70
4.2 Testing plan . . . 70

5 Evaluation **73**
5.1 Objective evaluation 73
5.2 Third-party feedback 73
5.3 Feedback evaluation 73
5.4 Improvements 73

1 Analysis

1.1 Problem statement

To form the perfect grouping for a group project within a school, many factors need to be taken into consideration to prevent students from feeling incapable of completing the project. These factors include, but are not limited to, skills of the students, skills required by the project, and number of students in the group.

This project aims to create a tool to aid teachers in ensuring the students are separated into groups where the students are best able to get along and combine their individual skills to achieve their project's aims. The user can input the data that they want to be taken into consideration by the program and it will output a set of groups. This is in the form of a table on a GUI - to present the results in a readable way - and an option to export the data.

This program will not find, assume, or generate information about the students or project, it will only work with the data inputted to assign a group of students to a project.

Teachers will benefit from this tool as they don't have to spend as much time or effort creating groups for the projects they set for their students, so they can spend their time and effort on helping and teaching the students instead. Also, when the students are working on their projects, they are more likely to be focused on their work, so require less oversight from the teacher.

Students will also benefit from this tool as they would be able to apply their skills to a project in a group of people they can enjoy time with, rather than being stuck on a project they have no interest in with people they do not get along with. This means they are more likely to enjoy their time at school, be able to get more work done, and therefore be able to learn more, as opposed to being unmotivated and distracted in a poorly made group.

1.2 Research

1.2.1 Sources of inspiration

1. Random group generator from ClassTools.net [Fig. 1]
   https://www.classtools.net/random-group-generator/

   - This tool is used frequently by my teachers to create random groups for group activities.

- The GUI is very simple to use as there is little data that needs to be entered, so can all fit down the left of the screen.
- The output is very easy to read as each group is visually separated using both coloured boxes and numbering. [Obj. 3b]
- However, this tool is too simple because it lacks any way to group with conditions, which is part of my scope, as they are purely randomised. This means that teams could quit often be unfairly balanced in terms of skill, or group students together that are known to not work well together. The teacher's only option to solve this is to re-roll until the output is suitable enough for the task manually. This gave me the idea for the algorithm to be the simulation of this re-rolling and selection of optimal outputs.

2. keamk [Fig. 2] [Fig. 3]
   https://www.keamk.com/random-team-generator
   - This tool creates groups that all have a balanced level of skill.
   - I like how the skill level is inputted as a rating between 1-5 as this is intuitive for the user.
   - I also like the extra details that are outputted, such as how many are in each group next to the group name and the skill levels next to each individual. [Obj. 3a]
   - Furthermore, the option to go back to make small changes to the data inputted and get new groupings was very quick and useful. [Obj. 1g]
   - However, this tool can only create balanced groups, not groups with different objectives as is in my scope, and there is no way to assign a certain number of participants to each group, as the program puts the same number (or one away) of participants in each group. [Obj. 2c-d]

These tools influenced me to reduce the complexity of my GUI, and gave me ideas such as to keep groups visually separate using colour as well as tables, adding the option to go back and edit the inputted data later, and some summary statistics to include with the outputs.

1.2.2 Algorithm

I have researched into representing students and projects into bipartite graphs, and then using matching algorithms such as the alternate path algorithm to join students to projects [Fig. 4]. However, this will only match one student to one project, not groups so is not suitable. I then researched applying the algorithm repeatedly to graphs with randomised students, but this means that some students will never be joined into a group purely based on chance and that the groups formed are such that each individual is best for the project, not that the overall group is best suited for the project, so teamwork will not

as good as it could be. I then researched using a grouping algorithm, such as https://help.feedbackfruits.com/hc/en-us/articles/23527084211346-Group-Formation-Algorithm-Explainer to create similar groups then using bipartite matching algorithms to match them. However this only creates groups with similar skill sets, not skill sets that vary based on the project.

I decided on a genetic algorithm where each individual is made up of a possible solution and the fitness score is an average between how suited each group is to its assigned project [Fig. 5]. The individuals with higher fitness scores are discarded and the lower ones are randomly selected to crossover to form new individuals. Some of these individuals formed may contain error such as the same student multiple times, or a missing student, so that must be corrected before continuing. Then, random mutations are applied to some individuals, which may involve swapping of students between groups, moving some students to an optional project, or moving some students to group that isn't full. New fitness scores are calculated and the process repeats a predetermined number of times [Fig. 9]. I chose this because genetic algorithms are very adaptable to varying conditions and constraints, such as the ones that the user chooses in this program. This will allow the program to test different combinations of projects and students, test different group sizes for a project, and swap in and out optional projects to find better and better solutions until it reaches a high-quality solution that fits all the user's needs. This fits the project scope far better than all previously researched algorithms.

1.2.3 Inputting data

For the inputting of data, I have decided that it can be inputted through txt and csv files [Obj. 1a], manually on the GUI [Obj. 1b], or any combination of those methods [Obj. 1c]. This is because the data for different individuals and different projects could be stored in different ways within the same system, and manual tweaks may be needed by the user to ensure all data is as is needed by the application. I will also include the option to import the data from an Excel spreadsheet at the request of my third party [Obj. 1b].

1.2.4 Accounts

I have decided against implementing an account system because I don't think it will provide any value to the program. Accounts are typically used for ensuring data can only be accessed by the users with the correct permissions. As this program will be designed to be run locally, all the data will be stored and only accessible on the machine the program was run on, so the OS can handle user security.

Another reason for accounts is allow users to access their data that has been stored by a third party, especially within a cloud to allow for data to be quickly transferred between devices. I will implement a manual solution to this by allowing users to export and import all the data the program stores and the user can handle that data transfer themselves.

1.2.5 Internal data storage

I chose to store the data about the students and projects in their own classes, that are aggregated within an 'individual' class. The student and project classes contain their names, skills and the project will have the recommended group size. The individual will contain a project object, student objects, and current fitness score as well as methods for calculating fitness scores, crossover with other individuals, and mutations. [Fig. 7]

This keeps the attributes and methods stored together in the program to make the code easier to read and understand by a programmer.

1.2.6 External data storage

[Obj. 4] I have researched into whether information about projects and students should be stored somewhere, and if so, how. I have concluded that they should be stored in relational databases that are accessible and modifiable by entering the details into the graphical interface, which are passed as parameters into SQL statements. These can then be loaded as a list of checkboxes that can be selected and deselected by the user for that run. [Obj. 1c]

I chose to store the data because being able to make minor changes to the data and then re-running the program quickly is a very valuable feature that could save users' time and energy as they would not have to go through the entire process of entering every piece of data just for small changes to some of the data. This feature is present in keamk's grouping tool, and I found it very useful in my testing.

I chose databases to store the data because I have more experience working with relational databases and SQL then any other form of data storage, such as in files. The data is structured so databases would allow for efficient access and modification of specific entries.

1.2.7 GUI library

Between the two main python GUI libraries, pygame and Tkinter, I chose Tkinter because of the larger library of classes of pre-made, customisable widgets that can be used more easily to produce a simple, easy-to-use tool, such as a dialog that allows users to graphically navigate through their files to find their data file [Fig.6].

On the other hand, with my prior experience with pygame, I have found it to be more suited towards building games and graphically-complex tools, which is not what I am aiming for in this project. I am aiming to have a GUI that is quick and simple to create and edit, and has user-friendly features for inputting, editing, and reading data. I also have more experience with Tkinter, which will reduce the amount of time I spend on the GUI and allow me to focus on developing and improving the grouping algorithm and user experience.

1.3 Dialogue

My third party is Dr Ed Horncastle, the lead teacher for the Exeter Mathematics Certificate (EMC) at my school, which is where groups of 3 or 4 students work together on research projects set by University academics or companies. He is suitable because he has to make these groups based on the skills of the students and match them to projects every year, so he has a thorough understanding of how this tool could work and would benefit from using this tool.

1.3.1 Initial interview

The following questions were asked during the research stage:

1. What is the current process for creating EMC groups?
    - All the metrics to consider are gathered into a spreadsheet and then groups are formed by hand.
    - Some projects don't end up getting groups assigned, especially those that have been set by a company that already has another project with an assigned group.
2. What are these metrics?
    - Students' own choice
    - There must be a mix of genders
    - The projects must be relevant to the A-level studies of that student. Someone who does not take computer science shouldn't be given a computer science based project
    - Those that work hard and those that laze off more easily should be mixed together. If this is their first project, this is inferred from GCSE results, otherwise it is based off the results of previous projects.
    - Special educational needs
    - Timetabling restrictions
    - Students do not get put into groups with others they have worked on projects with before
3. How are the student choices measured?
    - For some of the projects, the students give their top 3 choices of projects. Starting with the least popular project, the groups are formed by trying to minimise the sum of the positions the student's ranked the project in.
4. How is the data about the students and projects currently stored?
    - Excel spreadsheet
5. How would you prefer to input the data?

- Student choices from a Microsoft Form and all other data from a Excel spreadsheet

6. How much data and in what format do you think the data should be outputted?

    - In tables with the project name, student names and summaries of information about each group such as gender ratio, how many take computer science or physics, and how many have special education needs.
    - As a simplified register in alphabetical order

7. How useful would storing project/student details locally, that can be loaded when needed, be for you?

    - Very

8. Are there any other features that would be useful for this tool to include?

    - Some extra features that could be added is timetabling for what order the projects are presented in, and itinerary for group trips out of school.

After this, I decided to include Excel spreadsheets as an input method [Obj. 1b], include options for projects to not have groups assigned despite being a part of the algorithm [Obj. 1f], take genders into account [Obj. 2e], have summary statistics presented alongside the groups [Obj. 3a], have the option to view the groups without any extra information [Obj. 3b], store the data locally for re-use at another time [Obj. 4].

I decided not to include student choice, student personality, special education needs, timetabling restrictions, or preventing grouping the same students together across multiple as adding too many restrictions to the algorithm will make it take it longer to run to create a set of groups that best matches all of these restrictions. These are the restrictions that were lower on Dr Horncastle's priorities. I will also not include timetabling or scheduling as these deviate quite far from my project's scope.

1.3.2 Follow-up questions

The following questions were asked during the designing stage:

1. I have decided on using three factors to determine how "good" a group is: the number of students with at least one relevant skill to the tasked project; the number of skills required by the project that the group meets; whether or not there are at least two different genders in the group. Could you rate how important these factors are in your opinion, or whether you think any are equal?

- The students being able to contribute is twice as important as the other two factors combined
- Project skills are slightly more important than gender

1.4 Objectives

Objectives beginning with a * are extra objectives, only implemented if time allowed and are not required for the program to meet the scope of the problem.

1. Data can be entered into the program
   (a) Data can be entered manually into the GUI
   (b) Student and project data can be entered through file import
      i. File type can be csv
      ii. *File type can be txt
      iii. *File type can be a SQLite3 database
      iv. *File type can be xlsx
   (c) Data can be loaded from stores from previous uses of the program
   (d) Data can be entered through a combination of other methods
   (e) Student data must contain a name, gender, and skills
      i. Name must only contain alphanumeric characters and certain whitelisted characters
   (f) Project data must contain a name, maximum group size, minimum group size, whether or not the project must have a group or not, and skill requirements
      i. Name must only contain alphanumeric characters and certain whitelisted characters
      ii. Maximum group size must be greater than 0 and greater than minimum group size
      iii. Minimum group size must be greater than 0
   (g) Settings data must contain a target fitness, max number of iterations, and the size of the populations
      i. Target fitness but be between 0 and 1
      ii. Max iterations must be a positive integer, not including 0
      iii. Size of populations must be a positive integer greater than or equal to 4
   (h) Project and student data can be added, edited, and removed at any time before the genetic algorithm runs
   (i) At least one student, at least one project, target fitness, max iterations, and size of populations must all be entered/have values before the algorithm is run

2. Students must be grouped together and to a project
    (a) All students must be put into one group each
    (b) *All students must have at least one skill that will allow them to contribute to the project, when possible
    (c) Projects must not have more students assigned to it than the maximum limit inputted by the user
    (d) Projects must not have less students assigned to it than the minimum limit inputted by the user
    (e) *Groups must have a mix of genders, when possible
3. The resulting groups must be viewable
    (a) The user must have to option of coloured tables displayed on the GUI
        i. These tables contain the project name, student names, number of students, gender ratio, and skill range
        ii. Adjacent tables are coloured differently from each other
    (b) The user must have the option of a register layout displayed on the GUI
        i. Each project and its details are stored in separate boxes with a background distinct from its adjacent neighbours
        ii. Each box contains the project name, and the names of students in that group
        iii. Projects are displayed in alphabetical order
        iv. The students in each group are displayed in alphabetical order
    (c) Both above options can be exported to locally stored files
        i. File type can be csv
        ii. *File type can be txt
        iii. *File type can be xlsx
4. All data must be stored
    (a) The data about projects, students, and results must be stored in relational databases
    (b) The user must have the option to export the databases' contents to locally stored files
        i. File type can be csv
        ii. *File type can be txt
        iii. *File type can be xlsx
    (c) *The settings data must be stored as a JSON file
5. The design must be consistent across all pages

(a) Text is black with the Arial font

(b) The top of each page that displays data must feature a header

   i. The header contains text that provides a title for the data shown

   ii. The title text must be displayed on the left with a significantly larger font to the data shown

   iii. The header colour must be of hex code #FFDB32

   iv. The header may contain buttons, solely for navigation, on the right

(c) Data about different entities must be visually separate by keeping all the data about each single entity within individual boxes

(d) Buttons that perform a function that cannot be performed at that time must be greyed out, and feature text below stating how to enable the button

(e) Buttons that lead to a different page must feature the name of the page, or the relationship between the current page and next (e.g. "back")

(f) Up to 6 colours can be used to visually separate entities through colour. Red, orange, yellow, green, blue, purple. These colours must have an alpha value of between 10% and 50% to ensure the text remains legible

6. Users must be happy with the program

(a) 100% of users must be happy with navigating the program

(b) 100% of users must be happy with response times

(c) 100% of users must consider the groups outputted by the program to be suitable based on the provided data

(d) 100% of users must consider the methods of data entry to be convenient

(e) 100% of users must consider the methods of exporting data to be simple

(f) 100% of users must be able to read all data shown on the GUI

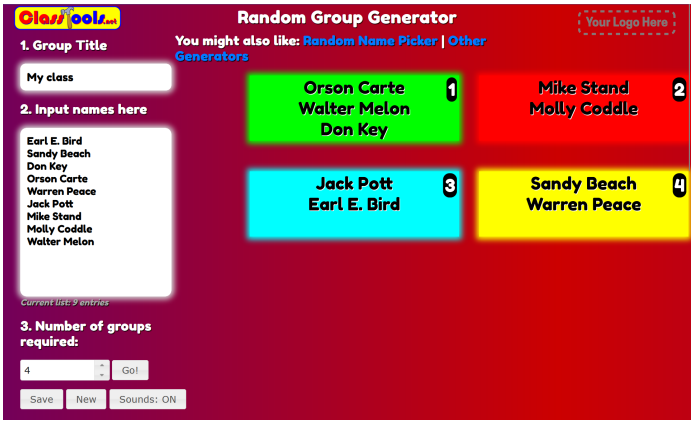


Figure 1: ClassTools's random group generator tool with inputs and outputs visible on the same page

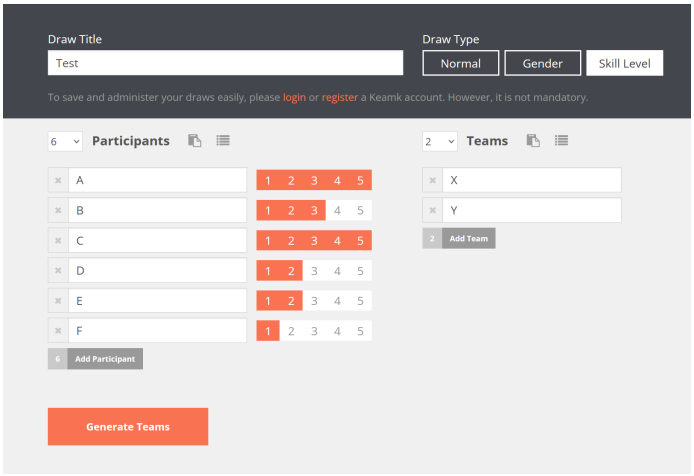


Figure 2: Keamk's menu for inputting data



Figure 3: Keamk's outputs

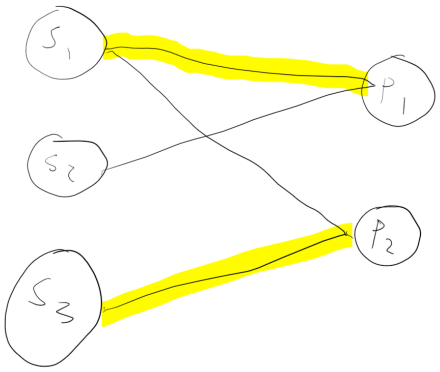


Figure 4: A bipartite graph with 3 student nodes on the left, 2 project nodes and the right, and some edges between them. A matching algorithm has been applied, with the matching highlighted in yellow. This has not met the scope because a student has not been matched.

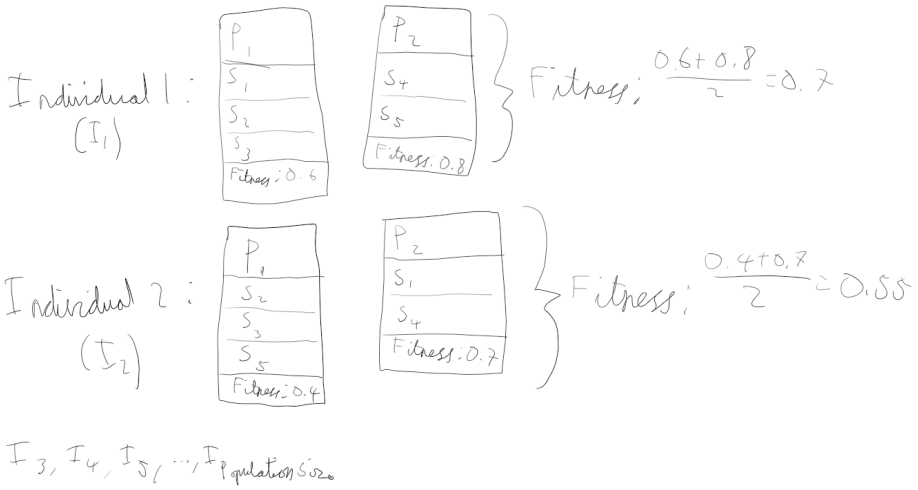


Figure 5: A sketch showing an example of a population, representing a set of possible solutions (individuals). Each individual represents a possible set of groups, and has a fitness score that represents how well every project's group matches the project's and user's requirements. This is calculated by averaging all the fitness scores of the groups it contains.

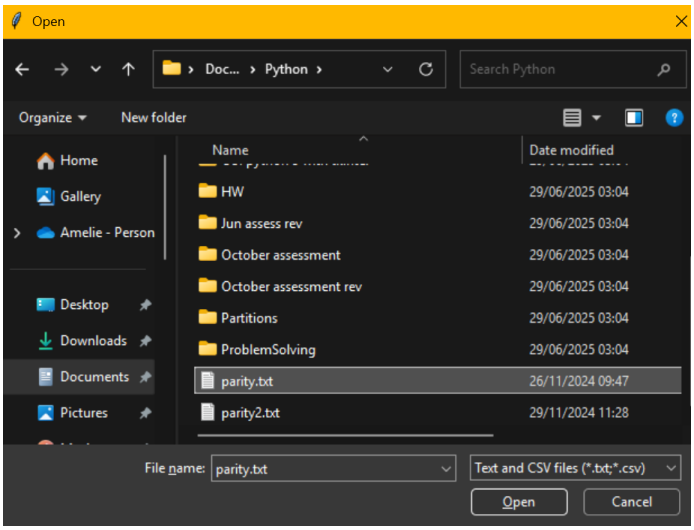


Figure 6: Tkinter's file dialog, configured to only show .txt and .csv files

1.5 Modelling

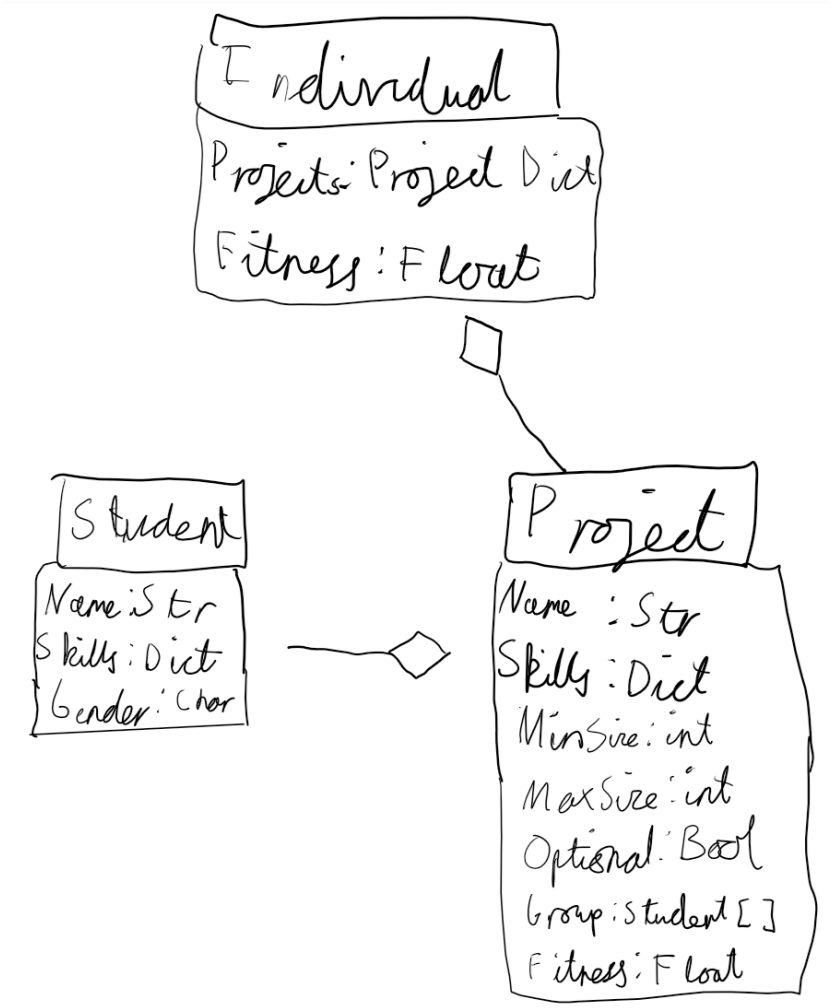


Figure 7: Class diagram showing relationships between the classes and their attributes used by the genetic algorithm.

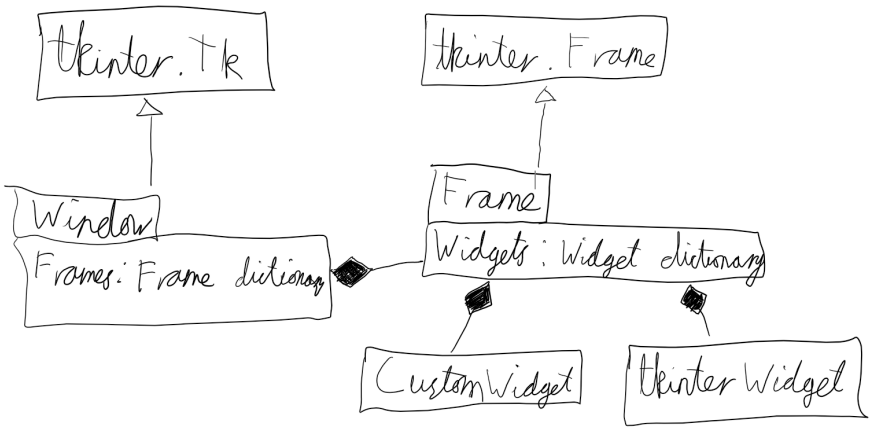


Figure 8: Class diagram showing relationships between the classes that form the GUI

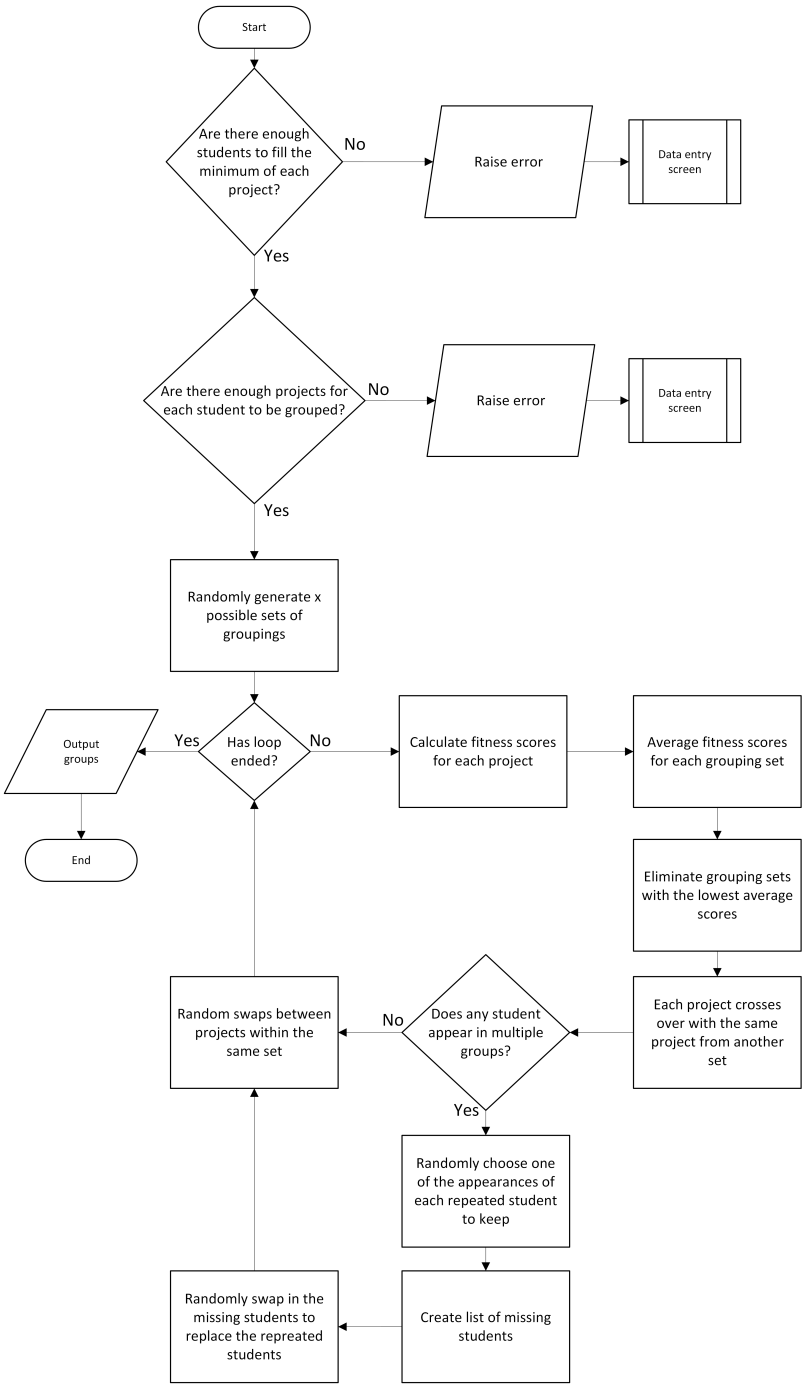


Figure 9: Flowchart showing the process for the genetic algorithm

2 Design

2.1 GUI

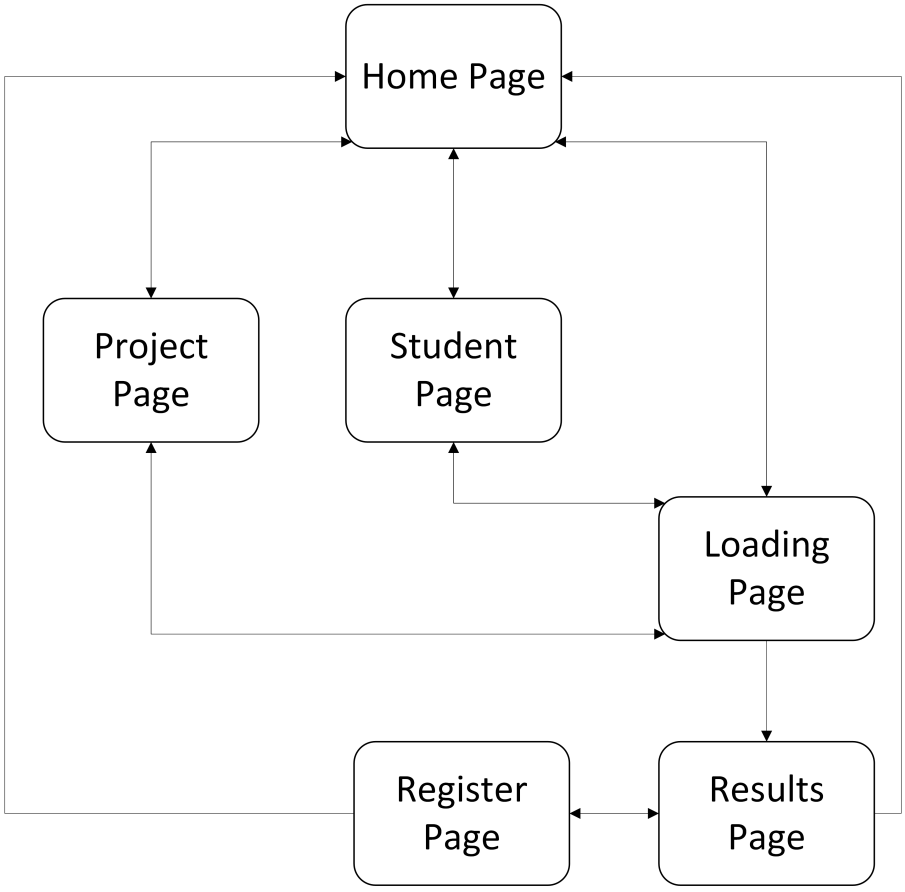


Figure 10: Page navigation diagram showing which pages can navigate to each other. Note: The student and project pages can only navigate to the previous page.

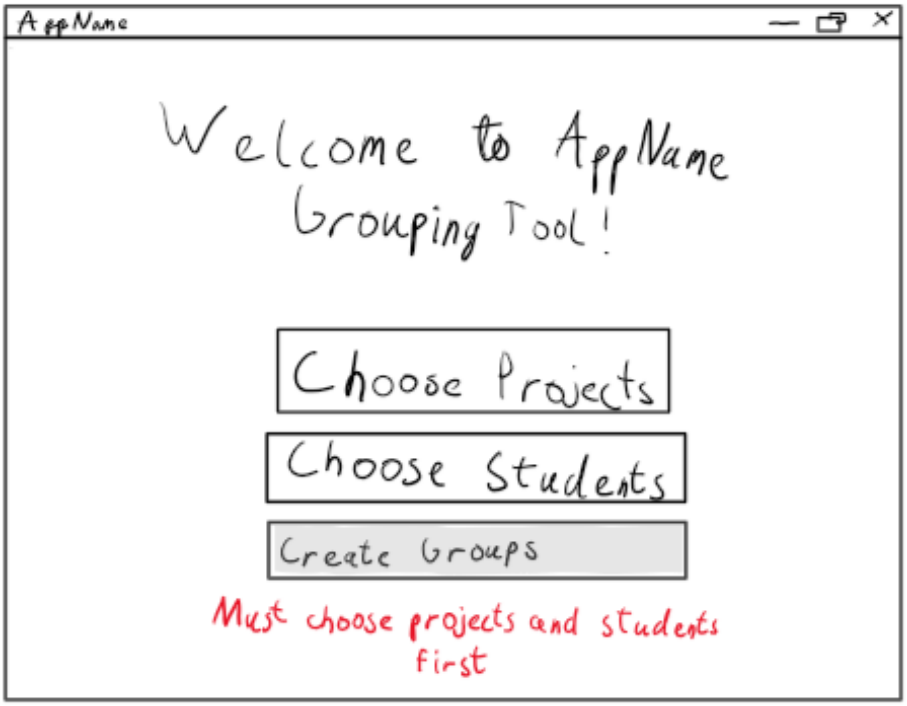


Figure 11: Sketch of the homepage layout that is the first page seen upon opening the app. The "Choose Projects" and "Choose Students" lead to the projects and students pages, respectively. The "Create Groups" button leads to the loading page and runs the grouping algorithm with the students and projects the user has selected. It is greyed out and not clickable unless at least one project and student has been selected [Obj. 1h].

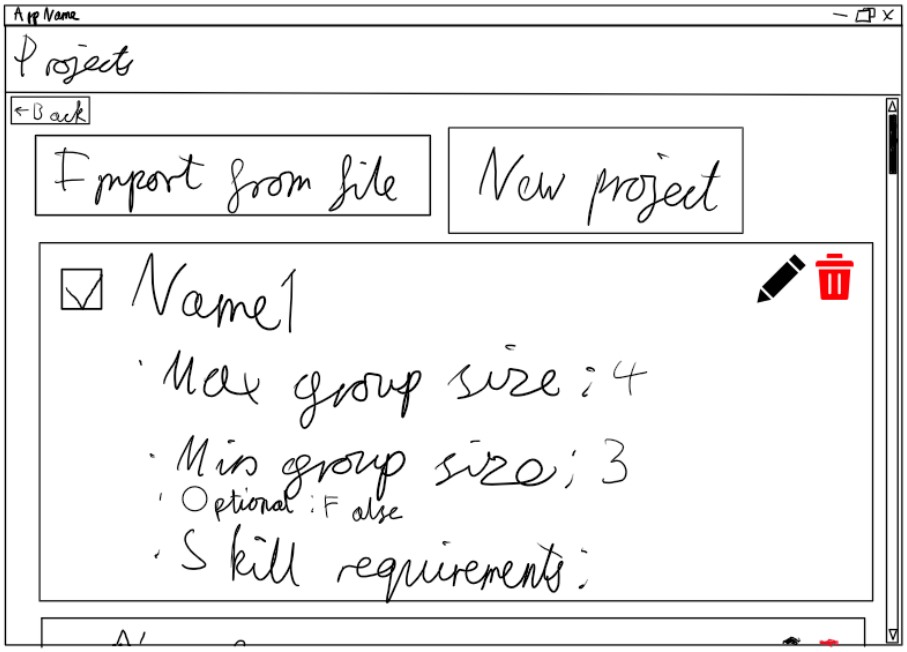


Figure 12: Sketch of the project page layout. The projects in the scrollable list are all those that are currently stored in the database [Obj. 1c]. There are options to import project data from supported files [Obj. 1b] and to manually enter project details at the top [Obj. 1a], which then get added to the database [Obj. 4] and the visual list [Obj. 1d]. The project details can also be edited or deleted from the database by pressing the buttons [Obj. 1g], which run parametrised SQL. Each project in the list features a checkbox, allowing users to select which of the projects in the database will be used in the grouping algorithm. Projects not selected will not have groups matched to them. This page also features a back button to go back to the home page.

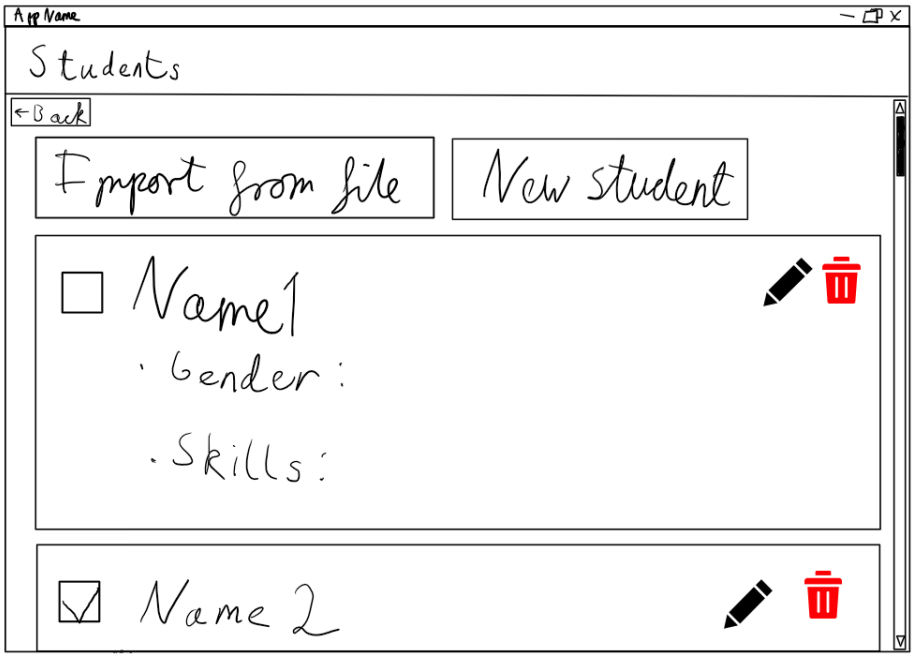


Figure 13: Sketch of the student page layout. The students in the scrollable list are all those that are currently stored in the database [Obj. 1c]. There are options to import student data from supported files [Obj. 1b] and to manually enter student details at the top [Obj. 1a], which then get added to the database [Obj. 4] and the visual list [Obj. 1d]. The student details can also be edited or deleted from the database by pressing the buttons [Obj. 1g], which run parametrised SQL. Each student in the list features a checkbox, allowing users to select which of the students in the database will be used in the grouping algorithm. Students not selected will not be put into groups. This page also features a back button to go back to the home page.

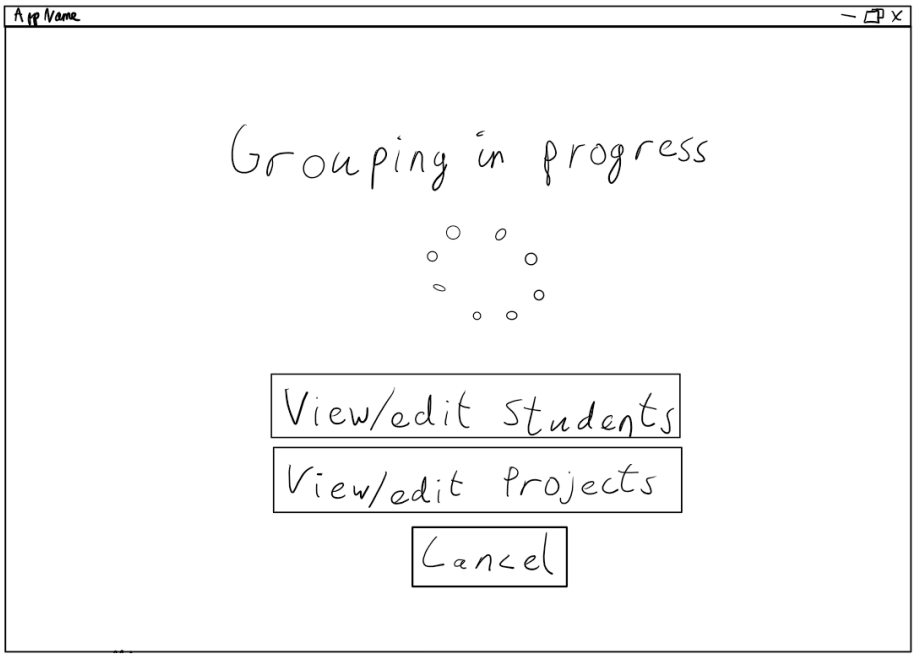


Figure 14: Sketch of the loading page layout. This page features a loading animation, and three buttons. The "View/edit Students" and "View/edit Projects" buttons lead to the Students and Projects pages, respectively [Obj. 1g]. The "Cancel" button ends the algorithm and leads to the home page. Once the algorithm has finished successfully, it moves onto the results page.

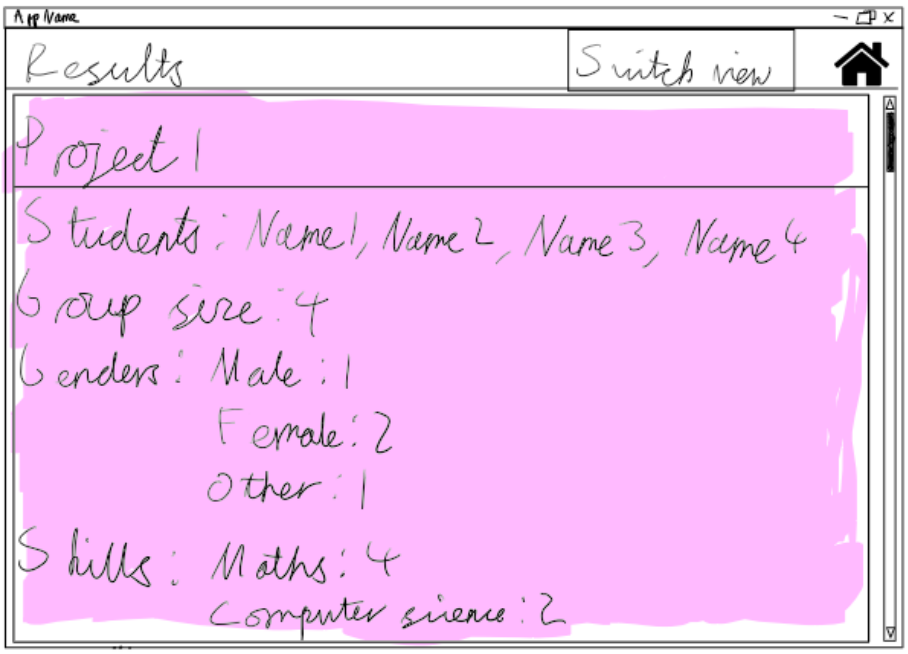


Figure 15: Sketch of the results page layout. It features a scrollable list of all the projects with the students grouped to it and summary statistics about the group, such as group size, gender ratio, and skills [Obj. 3a]. The "Switch view" leads to the register page and the home button leads to the home page.

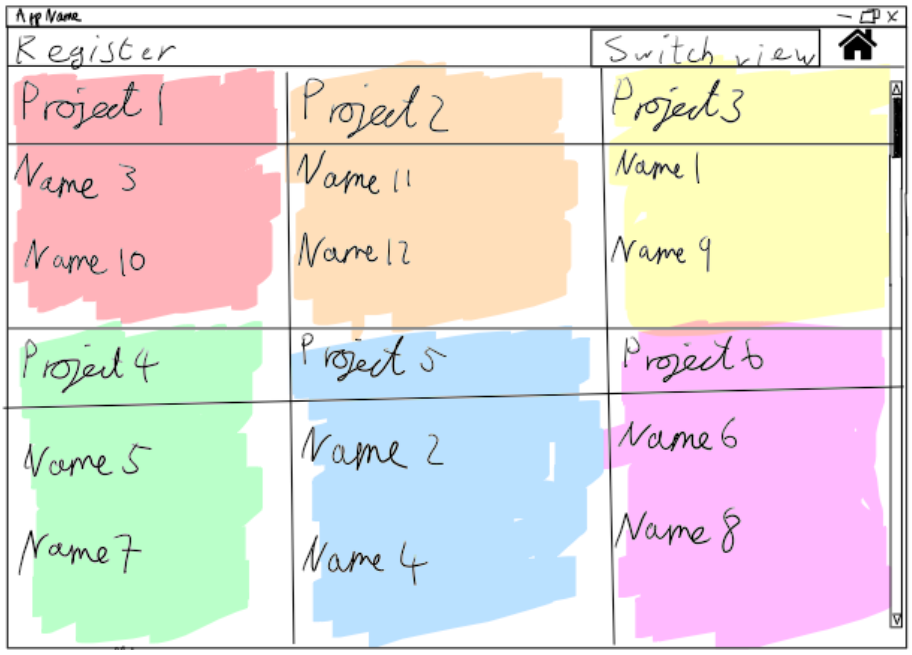


Figure 16: Sketch of the register page layout.

2.2 Databases

- Students(StudentID, Name, Gender)
- StudentSkills(StudentID, Skill)
- Projects(ProjectID, Name, MaxSize, MinSize, Optional)
- ProjectSkills(ProjectID, Skill)
- Results(ResultID, ProjectID, LocalTimeGenerated, EpochTimeGenerated)
- ResultStudentLink(ResultID, StudentID)

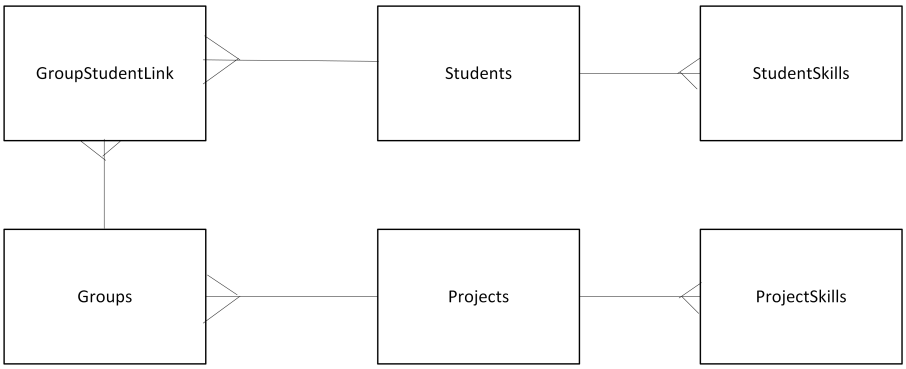


Figure 17: An entity relationship diagram showing the one-to-many relationships between the Students and Projects entities and the store of their skills to ensure all fields are atomic so the database is normalised.

2.3 Inputs and outputs

Table 1: An IPSO chart showing what data is inputted, how it is processed and stored, and how the results are outputted.

Input	Process	Storage	Output
Project details: - Name - Max group size - Min group size - Optional - Skill requirements	Validate input	Projects and ProjectSkills tables in database	Display on GUI [Fig. 12]

Student details: - Name - Gender - Skills	Validate input	Student and StudentSkills tables in database	Display on GUI [Fig. 13]
Settings: - Directory to export database to - Target fitness for genetic algorithm - Max iterations for genetic algorithm - Size of populations for genetic algorithm	Validate input	JSON file	NA
"Create Groups" button on homepage pressed	Run genetic algorithm[List. 1]	Classes for Student, Project, and Individual [Fig. 7]	Scrollable list of projects and groups [Fig. 15]

Table 2: Validations and restrictions on data entry

Data	Data Type	Validation and Restrictions
Projects		
Name	String	- Required
Max group size	Integer	- Required - Positive integers only
Min group size	Integer	- Required - Positive integers only - Must be less than or equal to 'Max group size'
Optional	Boolean	- Lack of entry is equivalent to entry of false
Skill requirements	String	- Entries separated by commas
Students		
Name	String	- Required
Gender	String	
Skills	String	- Entries separated by commas
Settings		

Directory to export database	String	- Must be a valid directory - Program must have read-/write permissions in that directory
Target fitness for genetic algorithm	Float	- Default is 0.9 - Must be between 0 and 1 inclusive
Max iterations for genetic algorithm	Integer	- Default is 50 - Must be a positive integer not including 0
Size of populations for genetic algorithm	Integer	- Default is 100 - Must be a positive integer greater than or equal to 4

2.4 Algorithm

2.4.1 Pseudocode

Listing 1: Pseudocode for the function that runs when on the loading page. It ensures it is possible to find groupings with the given requirements, then runs a genetic algorithm to find a good set of groupings. Some subroutines have flowcharts below.

```
1  TARGET_FITNESS = integer
2  MAX_ITERATIONS = integer
3  POPULATION_SIZE = integer
4
5
6  students = set of 'student' objects
7  projects = set of 'project' objects
8
9
10 #Ensure there aren't too many or too few students to meet
       ↪ requirements
11 minStudents = 0
12 maxStudents = 0
13
14 REPEAT for all projects:
15     maxStudents += project.maxSize
16     IF (not project.optional):
17         minStudents += project.minSize
18
19 #Check for too few students
20 IF (length of students < minStudents):
```

```
21      DisplayError("Error: Only {length of students} to
            ↪ fill a minimum of {minStudents} places.")
22      HomePage()
23
24  #Check for too many students
25  ELSE IF (length of students > maxStudents): #Checks for
        ↪ too many
26      DisplayError("{length of students} is too many to fit
            ↪  in {maxStudents} places.")
27      HomePage()
28
29
30  #Generate initial grouping sets and average fitness
        ↪ scores
31  population = list
32  REPEAT for i = 0 to POPULATION_SIZE - 1:
33      groups = GenInitialGroups(projects, students)
34      avgFit = (SUM fitness of each group)/(length of
            ↪ groups)
35      APPEND groups,avgFit to population
36
37  SORT population by avgFit
38
39  #Genetic algorithm
40  REPEAT for i = 1 to MAX_ITERATIONS:
41      #Ends early when one set of groupings is good enough
42      IF (highest avgFit > TARGET_FITNESS):
43          BREAK
44
45      #Eliminate the worst half of grouping sets
46      population = first (POPULATION_SIZE FLOORDIV 2) of
            ↪ population
47      new_pop = []
48
49      #Create next population
50      REPEAT for j = 1 to 90% of POPULATION_SIZE:
51          #Crossover random groupings
52          groups = Crossover(random group from population,
                ↪ random group from population, students)
53
54          #Ensure all students appear once
55          valid = verifyGroupings(groups)
56          IF (not valid):
57              fixGroupings(groups)
58
```

```
59          #Random mutations to prevent convergence towards
                ↪ a local best solution
60          Mutate(groups)
61
62          #Find the average fitness score for each new
                ↪ grouping
63          avgFit = (SUM fitness of each group)/(length of
                ↪ groups)
64          APPEND groups,avgFit to new_pop
65
66      #Combine the crossover with 10% elitism
67      population = new_pop + top 10% of population
68      SORT population by avgFit
69
70
71  return group with highest avgFit
```

2.4.2 Fitness algorithm

The fitness algorithm takes in the data about the current set of groupings and outputs a value between 0-1 that represents the suitability of the students to each project (0 being the worst possible groupings, 1 being absolutely perfect). This algorithm needs to allow the program to continually converge towards 1, guiding the program in the correct direction and helping it to reject bad solutions.

The factors that need to be considered by the algorithm are:

1. Each student should be able to contribute to their project [Obj. 2b] – they should have at least one skill that is demanded by the project
2. The demands of the project should be met by the students – the more skills are met by the students, the better the grouping
3. There should be a mix of genders [Obj. 2e] – there must be at least two different genders

These factors are of different levels of importance, so their contribution to the fitness value needs to be weighted. After talking with my third party, we decided on weights of 0.6, 0.25, and 0.15 respectively.

The fitness value for each group is then averaged to form a fitness for the whole set of groupings.

2.4.3 Diagrams

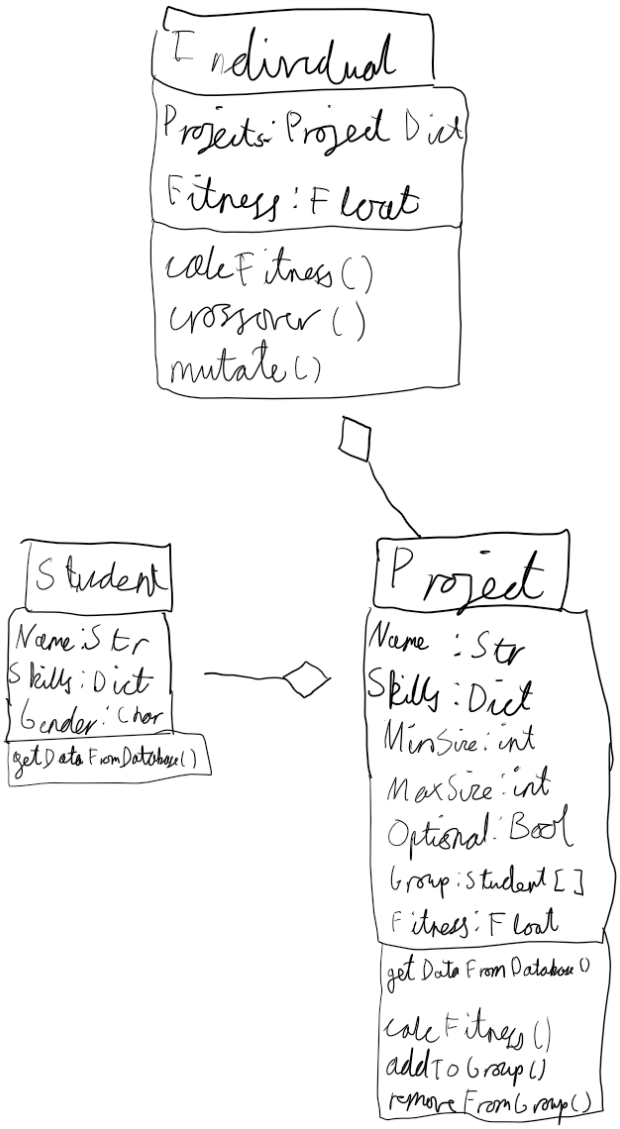


Figure 18: A class diagram showing the attributes and methods of the classes used by the genetic algorithm, and the relationships between each class.

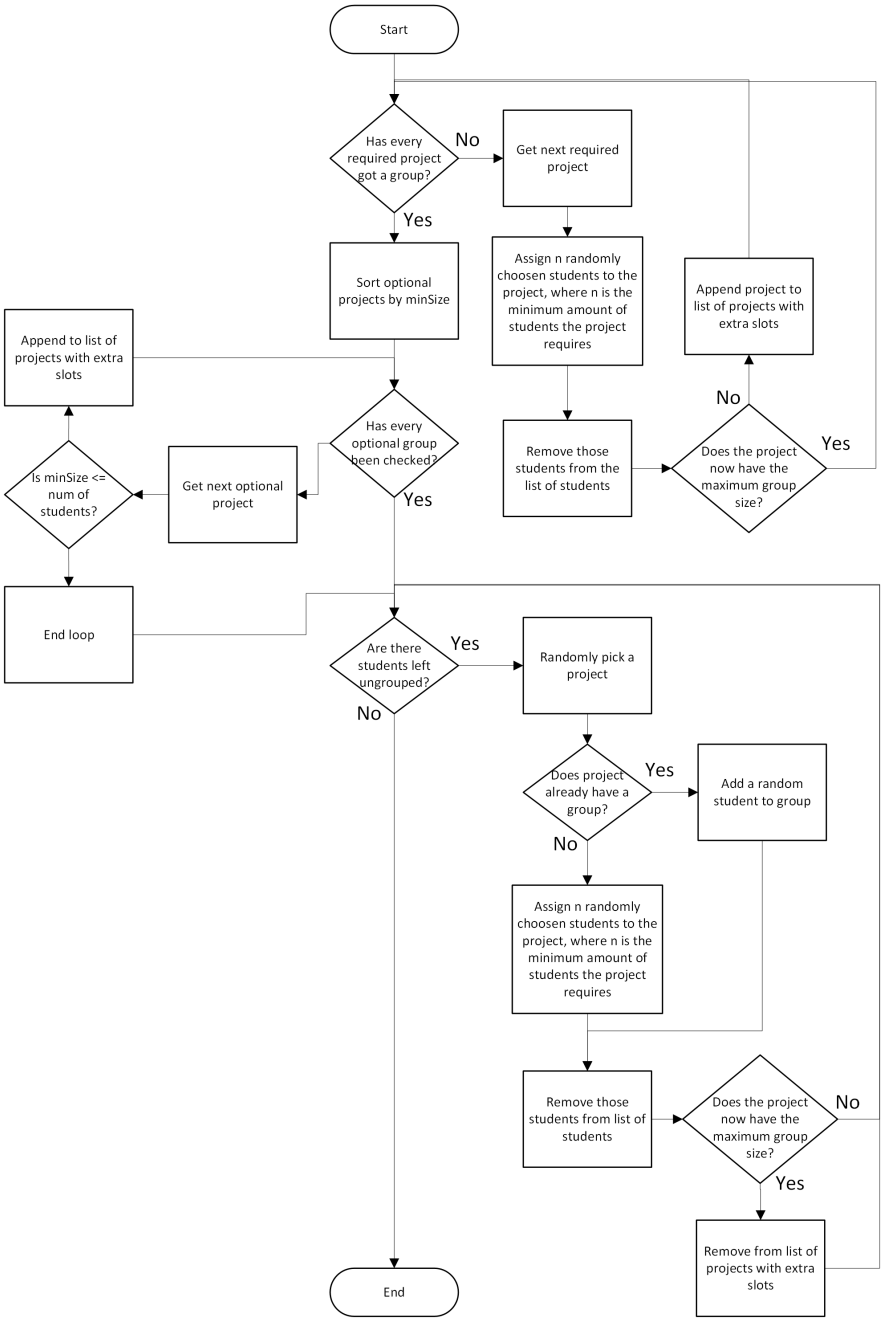


Figure 19: A flowchart for the GenInitialGroups subroutine on line 33 of the above pseudocode. It takes a list of all projects and a list of all students as parameters and returns an Individual object.

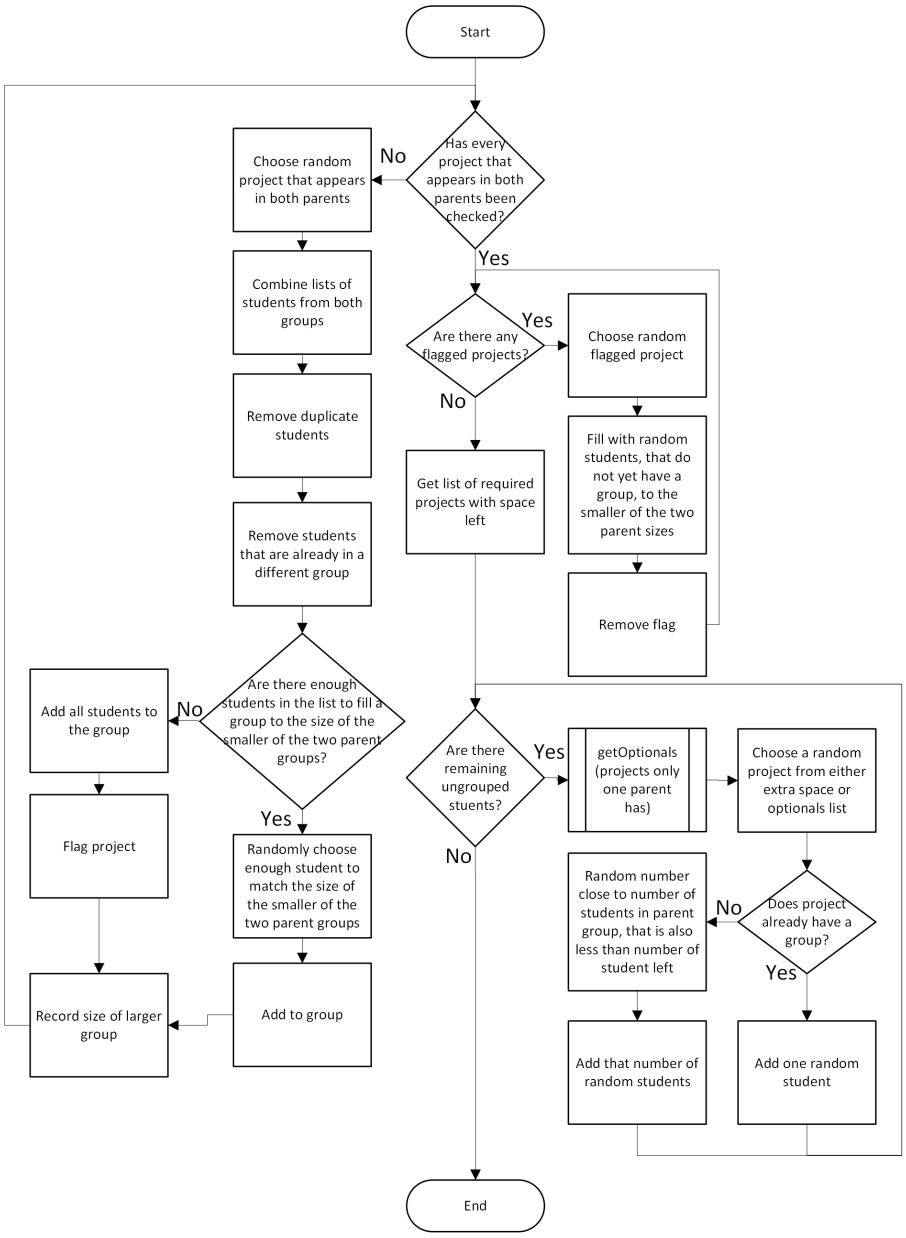


Figure 20: A flowchart for the Crossover method on line 52 of the above pseudocode and in the Individual class in the above class diagram. It takes two Individual objects, and the list of students, as parameters and returns an Individual object.

3 Technical solution

Contents

3.1 main.py 33
3.2 algoClasses.py 57
3.3 geneticAlgo.py 60
3.4 CreateTables.sql 65

main.py is the main file that is run by the user. It contains the GUI, and calls to the functions and classes in the other files when needed. All the pages outlined above [Fig. 10] have their own classes, whose objects are created dynamically and then stored in the window class. The page classes' attributes are mostly widgets, both tkinter widgets and custom widgets classes (such as Header, ScrollableFrame, SelectionBox, and ProjectBox), or the attributes store the result of user interaction, such as the state of the checkboxes on the project page. The methods all code for the processing and outputting the result of human interaction with the GUI.

algoClasses.py contains the Individual, Student, and Project classes for use by the genetic algorithm. These follow the design of the above class diagram [Fig. 18].They are stored in a separate file for re-use across multiple files.

geneticAlgo.py contains the code for genetic algorithm that calculates the groups. it follows the design of the above pseudocode [Listing 1]

Criteria

Criteria	Location
Complex data model in database (eg several interlinked tables)	CreateTables.sql file
Complex user-defined use of OOP model	-Inheritance. E.g. projectBox inherits from selectionBox -Composition e.g. ScrollableFrame in StudentPage -Polymorphism e.g. the tkraise method in HomePage -Abstract methods e.g. get_data in selectionBox
Cross-table parameterised SQL	Initialisation of ProjectPage
Dynamic generation of objects based on complex user-defined use of OOP model	The show_frame method of the Window class
Text files	write_csv method of the SettingsPage class

Writing and reading from files	-Writing e.g. write_csv method of the SettingsPage class -Reading soon
Dictionaries	frames attribute in Window class

3.1 main.py

```
import tkinter as tk
from tkinter import ttk, filedialog, messagebox
import sqlite3 as sql
import csv
import re
import geneticAlgo as ga
#from algoClasses import Student, Project, Individual

#=====GLOBAL CONSTANTS=====

BG_COLOUR="#FFFFFF"
HEADER_COLOUR="#FFDB32"
PASTEL_COLOURS=("#FFB3BB","#FFDFBA","#FFFFBA","#BAFFC9","#BAE1FF","#
    ↪ FFBAFF")
LARGE_FONT=("Arial",30)
DATABASE = "Code/database.db"
#SQLFILEHEADER="SQLite format 3"

#=====ROOT WINDOW=====
#Constants for styling included within

class Window(tk.Tk):
    def __init__(self):
        #Set up window
        tk.Tk.__init__(self)

        self.state('zoomed')
        self.geometry(f"{self.winfo_screenwidth()-10}x{self.
    ↪ winfo_screenheight()}")
        self.title('Student Grouping Tool')
        self.config(background=BG_COLOUR)

        #=====STYLES=====

        HEADER_BUTTON_STYLE=ttk.Style()
        HEADER_BUTTON_STYLE.configure("Header.TButton",background=
    ↪ HEADER_COLOUR)

        #================
        #Dictionary that stores all loaded pages
```

```
38          self.frames={}
39          #Load the saved settings
40          self.load_settings()
41          #Load the homepage first
42          self.show_frame(HomePage)
43
44      #Method for changing currently open page
45      def show_frame(self, f,*args,**kwargs):
46          #If the page is already loaded, show it
47          if f in self.frames.keys():
48              frame = self.frames[f]
49          #Otherwise, load it then show it
50          else:
51              frame = f(self)
52              self.frames[f] = frame
53              frame.place(relx=0,rely=0,relwidth=1,relheight=1)
54          frame.tkraise(*args,**kwargs)
55
56      def load_settings(self):
57          #Checks for settings.json, if exists load data. If not create
    ↪ settings.json with the default values. Regardless after load
    ↪ settings page into the dict with the required settings as params.
58          frame=SettingsPage(self,0.9,50,100)
59          self.frames[SettingsPage]=frame
60          frame.place(relx=0,rely=0,relwidth=1,relheight=1)
61          return
62
63  #=====PAGES=====
64
65  class HomePage(tk.Frame):
66      def __init__(self,parent):
67          #Set up home page layout
68          tk.Frame.__init__(self, parent)
69
70          self.settingsButton = ttk.Button(self,text="Settings",command=
    ↪ lambda:parent.show_frame(SettingsPage))
71          self.settingsButton.pack()
72
73          self.label = ttk.Label(self, text="Welcome to the Student
    ↪ Grouping Tool!", font=LARGE_FONT)
74          self.label.pack()
75
76          self.projectButton = ttk.Button(self,text="Choose Projects",
    ↪ command=lambda:parent.show_frame(ProjectPage))
77          self.projectButton.pack()
78
79          self.studentButton = ttk.Button(self,text="Choose Students",
    ↪ command=lambda:parent.show_frame(StudentPage))
80          self.studentButton.pack()
81
```

```
82          self.runButton = ttk.Button(self,text="Create Groups",command=
    ↪ self.get_data_for_algo,state="disabled")
83          self.runButton.pack()
84
85          self.warning = ttk.Label(self,text="Must choose projects and
    ↪ students first")
86
87
88      #Method override that runs once everytime the page is opened
89      def tkraise(self,*args,**kwargs):
90          tk.Frame.tkraise(self)
91
92          #If at least one student and one project has been selected, allow
    ↪  the run button to be clickable and remove the warning message
93          if ProjectPage in self.master.frames and StudentPage in self.
    ↪ master.frames and self.master.frames[ProjectPage].
    ↪ check_for_selection() and self.master.frames[StudentPage].
    ↪ check_for_selection():
94              self.runButton["state"]="enabled"
95              self.warning.pack_forget()
96
97          #Otherwise, disable the run button and display the warning
    ↪ message
98          else:
99              self.runButton["state"]="disabled"
100             self.warning.pack()
101
102     def get_data_for_algo(self,*args,**kwargs):
103         database=DATABASE
104         projects=self.master.frames[ProjectPage].get_selected()
105         students=self.master.frames[StudentPage].get_selected()
106         settings=self.master.frames[SettingsPage].get_settings()
107         fitness=settings["fitness"]
108         iterations=settings["iterations"]
109         popSize=settings["popSize"]
110
111         self.master.show_frame(LoadingPage,database,students,projects,
    ↪ fitness,iterations,popSize)
112
113
114
115
116
117 class ProjectPage(tk.Frame):
118     def __init__(self,parent):
119         #Set up the project page layout
120         tk.Frame.__init__(self,parent)
121
122         self.header=Header(self,"Projects",HEADER_COLOUR)
123         self.header.pack(fill="x")
```

```
124
125         self.backButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage))
126         self.backButton.pack(side="right")
127
128         self.importButton=ttk.Button(self,text="Import from file",command
    ↪ =self.import_projects)
129         self.importButton.pack()
130
131         self.addButton=ttk.Button(self,text="New project",command=lambda:
    ↪ self.create_entry_window("New"))
132         self.addButton.pack()
133
134         #self.separator=ttk.Separator(self,orient="horizontal")
135         #self.separator.pack(fill="x",expand=True)
136
137         self.boxesFrame=ScrollableFrame(self)
138         self.boxesFrame.canvas.pack(fill="both",expand=True)
139         self.update()
140
141         #Get and show project data from the database
142         self.projectBoxes={}
143         self.fetch_from_db(DATABASE)
144
145
146     def fetch_from_db(self,path):
147         #Fetch all project data from a database
148         query="SELECT * from Projects LEFT JOIN ProjectSkills using (
    ↪ ProjectID)"
149         with sql.connect(path) as conn:
150             cur = conn.cursor()
151             cur.execute(query)
152             projects=cur.fetchall()
153
154             for project in projects:
155                 ID=project[0]
156                 #When a project has multiple skills, the ID is repeated
    ↪ in the recieved tuple for each skill
157                 if ID in self.projectBoxes:
158                     #The next skill is appended to the existing list of
    ↪ skills for this ID
159                     data=self.projectBoxes[ID].get_data()
160
161                     skills=data["skills"]
162                     skills.append(project[5])
163                     self.projectBoxes[ID].edit_data(skills=skills)
164
165                 #If the project has not been added to the dictionary yet,
    ↪  add it
166                 else:
```

```
167                     #Unpack the data, keeping data that should not be of
    ↪ type string spearate
168                     *data,optional,skills=project
169                     #Convert to correct data type
170                     optional=bool(optional)
171                     skills=[skills]
172                     #Store data and display data on UI
173                     self.projectBoxes[ID]=projectBox(self.boxesFrame,self
    ↪ ,*data,optional,skills)
174                     self.projectBoxes[ID].pack()
175
176
177     #Method override as a work-around for a scrolling bug
178     def update(self):
179         self.master.state('normal')
180         tk.Frame.update(self)
181         self.master.state('zoomed')
182
183     #Iterates through each project, returning true as soon as it finds a
    ↪ selected project. Returns false when no project is selected.
184     def check_for_selection(self):
185         for box in self.projectBoxes:
186             if self.projectBoxes[box].check_selection():
187                 return True
188         return False
189
190     def get_selected(self):
191         sel=set()
192         for box in self.projectBoxes:
193             if self.projectBoxes[box].check_selection():
194                 sel.add(box)
195         return sel
196
197
198     def create_entry_window(self,mode,**kwargs):
199         name=tk.StringVar()
200         maxSize=tk.StringVar()
201         minSize=tk.StringVar()
202         optional=tk.BooleanVar()
203         skills=tk.StringVar()
204         data={"name":name,"maxSize":maxSize,"minSize":minSize,"optional":
    ↪ optional,"skills":skills}
205
206         if mode == "Edit":
207             popupWin=entryPopupWin(self,"Edit Project")
208             name.set(kwargs["name"])
209             maxSize.set(kwargs["maxSize"])
210             minSize.set(kwargs["minSize"])
211             optional.set(kwargs["optional"])
212             skills.set(", ".join(kwargs["skills"]))
```

```
213             data["ID"]=kwargs["ID"]
214         else:
215             popupWin=entryPopupWin(self,"New project")
216 
217         popupWin.add_string_entry("Name:",name)
218 
219         popupWin.add_int_entry("Maximum size allowed for this project's
    ↪ group:",maxSize)
220 
221         popupWin.add_int_entry("Minimum size allowed for this project's
    ↪ group:",minSize)
222 
223         popupWin.add_checkbox("Optional:",optional)
224 
225         popupWin.add_string_entry("Skills, each one separated by a comma:
    ↪ ",skills)
226 
227         popupWin.add_save_cancel(popupWin,mode,**data)
228 
229     def save_entry(self,window,mode,**kwargs):
230         name=kwargs["name"].get()
231         maxSize=kwargs["maxSize"].get()
232         minSize=kwargs["minSize"].get()
233         optional=kwargs["optional"].get()
234         skills=kwargs["skills"].get()
235 
236         #Validation to ensure all required entries are filled
237         if name != "" and maxSize != "" and minSize != "" and skills != "
    ↪ ":
238 
239             maxSize = int(maxSize)
240             minSize = int(minSize)
241 
242             #Separate skills into a list, remove whitespace, and add to a
    ↪  set to remove duplicates
243             skillList=skills.split(",")
244             skills=set()
245             for i in range(len(skillList)):
246                 skills.add(skillList[i].strip())
247 
248             if mode == "Edit":
249                 self.edit_project(kwargs["ID"],name,maxSize,minSize,
    ↪ optional,skills)
250             else:
251                 self.new_project(name,maxSize,minSize,optional,skills)
252 
253             #Destroy the popup window
254             window.destroy()
255 
256 
```

```
257     def import_projects(self):
258         def importtxt(path):
259             file=open(path,mode="r")
260         def importcsv(path):
261             file=open(path,newline='')
262             reader=csv.reader(file)
263 
264 
265         def importdb(path):
266             return
267         #Show an "Open" dialog box and return file directory
268         path = filedialog.askopenfilename() #Add excel
269         #Validation to ensure the user selected a valid file type
270         if path != None:
271             match path[-4:]:
272                 case ".txt":
273                     importtxt(path)
274                 case ".csv":
275                     importcsv(path)
276                 case _:
277                     if checkForDB(path):
278                         importdb(path)
279                     else:
280                         messagebox.showwarning(title="Import Warning",
    ↪ message="File type unrecognised!")
281 
282         else:
283             messagebox.showwarning(title="Import Warning", message="No
    ↪ file selected!")
284 
285 
286 
287     #Add the inputted data to the Project and Project Skills tables in
    ↪ the database
288     def new_project(self,name,maxSize,minSize,optional,skills):
289         mainquery="INSERT INTO Projects (Name, MaxSize, MinSize, Optional
    ↪ ) VALUES (?,?,?,?)"
290         skillsquery="INSERT INTO ProjectSkills (ProjectID, Skill) VALUES
    ↪ (?,?)"
291         with sql.connect(DATABASE) as conn:
292             cur = conn.cursor()
293 
294             cur.execute(mainquery,(name,maxSize,minSize,optional))
295 
296             #Fetch the autogenerated ID for this student
297             ID=cur.lastrowid
298 
299             for skill in skills:
300                 cur.execute(skillsquery,(ID, skill))
301 
```

```
302             conn.commit()
303 
304         #Display new project on the GUI
305         self.projectBoxes[ID]=projectBox(self.boxesFrame,self,ID,name,
    ↪ maxSize,minSize,optional,skills)
306         self.projectBoxes[ID].pack()
307 
308         #Work-around for a scrollbar bug
309         self.update()
310 
311     def edit_project(self,ID,name,maxSize,minSize,optional,skills):
312         edited=False
313         box=self.projectBoxes[ID]
314         oldData=box.get_data()
315 
316         #Only updates if the new data is different from the old data
317         with sql.connect(DATABASE) as conn:
318             cur = conn.cursor()
319 
320             if name != oldData["name"]:
321                 edited=True
322 
323                 query = "UPDATE Projects SET Name = ? WHERE ProjectID = ?
    ↪ "
324                 cur.execute(query,(name,ID))
325 
326                 box.edit_data(name=name)
327 
328             if maxSize != oldData["maxSize"]:
329                 edited=True
330 
331                 query="UPDATE Projects SET MaxSize = ? WHERE ProjectID =
    ↪ ?"
332                 cur.execute(query,(maxSize,ID))
333 
334                 box.edit_data(maxSize=maxSize)
335 
336             if minSize != oldData["minSize"]:
337                 edited=True
338 
339                 query="UPDATE Projects SET MinSize = ? WHERE ProjectID =
    ↪ ?"
340                 cur.execute(query,(minSize,ID))
341 
342                 box.edit_data(minSize=minSize)
343 
344             if optional != oldData["optional"]:
345                 edited=True
346 
```

```
347                 query="UPDATE Projects SET Optional = ? WHERE ProjectID =
    ↪  ?"
348                 cur.execute(query,(optional,ID))
349 
350                 box.edit_data(optional=optional)
351 
352             if skills != oldData["skills"]:
353                 edited=True
354 
355                 cur.execute("DELETE FROM ProjectSkills WHERE ProjectID =
    ↪ ?",(ID,))
356                 query = "INSERT INTO ProjectSkills (ProjectID, Skill)
    ↪ VALUES (?,?)"
357                 for skill in skills:
358                     cur.execute(query,(ID,skill))
359 
360                 box.edit_data(skills=skills)
361 
362             if edited:
363                 self.update()
364                 conn.commit()
365 
366     def editBox(self,boxID):
367         box=self.projectBoxes[boxID]
368         data=box.get_data()
369         self.create_entry_window("Edit",ID=data["ID"],name=data["name"],
    ↪ maxSize=data["maxSize"],minSize=data["minSize"], optional=data["
    ↪ optional"], skills=data["skills"])
370 
371     def deleteBox(self,boxID):
372         #Delete from database
373         query1 = "DELETE FROM Projects WHERE ProjectID=?"
374         query2 = "DELETE FROM ProjectSkills WHERE ProjectID=?"
375         with sql.connect(DATABASE) as conn:
376             cur = conn.cursor()
377             cur.execute(query1,(boxID,))
378             cur.execute(query2,(boxID,))
379             conn.commit()
380         #Delete from UI
381         self.projectBoxes[boxID].pack_forget()
382         #Delete from internal store
383         del self.projectBoxes[boxID]
384 
385         self.update()
386 
387 
388 
389 class StudentPage(tk.Frame):
390     def __init__(self,parent):
391         #Set up student page layout
```

```
392         tk.Frame.__init__(self, parent)
393 
394         self.header=Header(self,"Students",HEADER_COLOUR)
395         self.header.pack(fill="x")
396 
397         self.backButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage))
398         self.backButton.pack(side="right")
399 
400         self.importButton=ttk.Button(self,text="Import from file",command
    ↪ =self.import_students)
401         self.importButton.pack()
402 
403         self.addButton=ttk.Button(self,text="New student",command=lambda:
    ↪ self.create_entry_window("New"))
404         self.addButton.pack()
405 
406         self.boxesFrame=ScrollableFrame(self)
407         self.boxesFrame.canvas.pack(fill="both",expand=True)
408         self.update()
409 
410         self.studentBoxes={}
411 
412         #Fetch all student data from the database
413         studentquery="SELECT * from Students LEFT JOIN StudentSkills
    ↪ using (StudentID)"
414         with sql.connect(DATABASE) as conn:
415             cur = conn.cursor()
416             cur.execute(studentquery)
417             students=cur.fetchall()
418 
419             for student in students:
420                 ID=student[0]
421                 #When a student has multiple skills, the ID is repeated
    ↪ in the recieved tuple for each skill
422                 if ID in self.studentBoxes:
423                     #The next skill is appended to the existing list of
    ↪ skills for this ID
424                     data=self.studentBoxes[ID].get_data()
425                     skills=data["skills"]
426                     skills.append(student[3])
427                     self.studentBoxes[ID].edit_data(skills=skills)
428                 #If the student has not been added to the dictionary yet,
    ↪  add it
429                 else:
430                     #Unpack the data, keeping data that should not be of
    ↪ type string spearate
431                     *data,skills=student
432                     #Convert to correct data type
433                     skills=[skills]
```

```
434                     #Store data and display on UI
435                     self.studentBoxes[ID]=studentBox(self.boxesFrame,self
    ↪ ,*data,skills)
436                     self.studentBoxes[ID].pack()
437
438     #Method override as a work-around for a scrolling bug
439     def update(self):
440         self.master.state('normal')
441         tk.Frame.update(self)
442         self.master.state('zoomed')
443
444     def check_for_selection(self):
445         for box in self.studentBoxes:
446             if self.studentBoxes[box].check_selection():
447                 return True
448         return False
449
450     def get_selected(self):
451         sel=set()
452         for box in self.studentBoxes:
453             if self.studentBoxes[box].check_selection():
454                 sel.add(box)
455         return sel
456
457     def import_students(self):
458         #Show an "Open" dialog box and return an open file object in read
    ↪  mode
459         file = filedialog.askopenfile(mode='r',filetypes=[("Supported
    ↪ file types",".txt .csv")]) #Add excel
460         #Validation to ensure the user selected a file
461         if file == None:
462             return
463
464         #<<<-------------------------------------------
465
466         file.close()
467         return
468
469     def create_entry_window(self,mode,**kwargs):
470         name=tk.StringVar()
471         gender=tk.StringVar()
472         skills=tk.StringVar()
473         data={"name":name,"gender":gender,"skills":skills}
474
475         if mode == "Edit":
476             popupWin=entryPopupWin(self, "Edit student")
477             name.set(kwargs["name"])
478             gender.set(kwargs["gender"])
479             skills.set(", ".join(kwargs["skills"]))
480             data["ID"]=kwargs["ID"]
```

```
481             else:
482                 popupWin=entryPopupWin(self,"New student")
483 
484             popupWin.add_string_entry("Name:", name)
485 
486             popupWin.add_string_entry("Gender:",gender)
487 
488             popupWin.add_string_entry("Skills, each one separated by a comma:
    ↪ ",skills)
489 
490             popupWin.add_save_cancel(popupWin,mode,**data)
491 
492         def save_entry(self,window,mode,**kwargs):
493             name=kwargs["name"].get()
494             gender=kwargs["gender"].get()
495             skills=kwargs["skills"].get()
496 
497             #Validation to ensure all required entries are filled
498             if name != "" and gender != "" and skills != "":
499 
500                 #Separate skills into a list, remove whitespace, add to set
    ↪ to remove duplicates
501                 skillList=skills.split(",")
502                 skills=set()
503                 for i in range(len(skillList)):
504                     skills.add(skillList[i].strip())
505 
506                 if mode == "Edit":
507                     self.edit_student(kwargs["ID"],name,gender,skills)
508                 else:
509                     self.new_student(name,gender,skills)
510 
511                 #Destroy the popup window
512                 window.destroy()
513 
514 
515         #Add the inputted data to the Student and StudentSkills tables in the
    ↪  database
516         def new_student(self,name,gender,skills):
517             mainquery="INSERT INTO Students (Name, Gender) VALUES (?,?)"
518             skillsquery="INSERT INTO StudentSkills (StudentID, Skill) VALUES
    ↪ (?,?)"
519             with sql.connect(DATABASE) as conn:
520                 cur = conn.cursor()
521 
522                 cur.execute(mainquery,(name,gender))
523 
524                 #Fetch the autogenerated ID for this student
525                 ID=cur.lastrowid
526 
```

```
527             for skill in skills:
528                 cur.execute(skillsquery,(ID, skill))
529 
530             conn.commit()
531 
532         #Display new student on the GUI
533         self.studentBoxes[ID]=studentBox(self.boxesFrame,self,ID,name,
    ↪ gender,skills)
534         self.studentBoxes[ID].pack()
535 
536         #Scroll bar bug work-around
537         self.update()
538 
539 
540     def edit_student(self,ID,name,gender,skills):
541         edited=False
542 
543         box=self.studentBoxes[ID]
544         oldData=box.get_data()
545 
546         #Only updates if the new data is different from the old data
547         with sql.connect(DATABASE) as conn:
548             cur = conn.cursor()
549 
550             if name != oldData["name"]:
551                 edited=True
552 
553                 query = "UPDATE Students SET Name = ? WHERE StudentID = ?
    ↪ "
554                 cur.execute(query,(name,ID))
555 
556                 box.edit_data(name=name)
557 
558             if gender != oldData["gender"]:
559                 edited=True
560 
561                 query="UPDATE Students SET Gender = ? WHERE StudentID = ?
    ↪ "
562                 cur.execute(query,(gender,ID))
563 
564                 box.edit_data(gender=gender)
565 
566             if skills != oldData["skills"]:
567                 edited=True
568 
569                 cur.execute("DELETE FROM StudentSkills WHERE StudentID =
    ↪ ?",(ID,))
570                 query = "INSERT INTO StudentSkills (StudentID, Skill)
    ↪ VALUES (?,?)"
571                 for skill in skills:
```

```
572                     cur.execute(query,(ID,skill))
573
574                 box.edit_data(skills=skills)
575
576             if edited:
577                 self.update()
578                 conn.commit()
579
580     def editBox(self,boxID):
581         box=self.studentBoxes[boxID]
582         data=box.get_data()
583         self.create_entry_window("Edit", ID=data["ID"],name=data["name"],
    ↪ gender=data["gender"],skills=data["skills"])
584
585     def deleteBox(self,boxID):
586         #Delete from database
587         query1 = "DELETE FROM Students WHERE StudentID=?"
588         query2 = "DELETE FROM StudentSkills WHERE StudentID=?"
589         with sql.connect(DATABASE) as conn:
590             cur = conn.cursor()
591             cur.execute(query1,(boxID,))
592             cur.execute(query2,(boxID,))
593             conn.commit()
594         #Delete from UI
595         self.studentBoxes[boxID].pack_forget()
596         #Delete from internal store
597         del self.studentBoxes[boxID]
598
599         self.update()
600
601
602
603 class LoadingPage(tk.Frame):
604     def __init__(self,parent):
605         #Set up loading page layout
606         tk.Frame.__init__(self, parent)
607
608         self.label = ttk.Label(self, text="Grouping in progress...")
609         self.label.pack()
610
611         self.cancelButton=ttk.Button(self,text="Cancel",command=lambda:
    ↪ parent.show_frame(HomePage))
612         self.cancelButton.pack()
613
614     #Method override that runs once everytime the page is opened
615     def tkraise(self, database, studentIDs, projectIDs, tgtFit, maxIters,
    ↪  popSize,*args,**kwargs):
616         tk.Frame.tkraise(self)
617         results=ga.run(database,studentIDs,projectIDs,tgtFit,maxIters,
    ↪ popSize)
```

```
618         if results["status"]==-1:
619             messagebox.showerror("Error",results["error"])
620         elif results["status"]==0:
621             results=results["results"]
622             self.master.show_frame(ResultsPage,results)
623         else:
624             messagebox.showerror("Error","An unknown eror occured")
625 
626 
627 class ResultsPage(tk.Frame):
628     def __init__(self,parent):
629         #Set up results page layout
630         tk.Frame.__init__(self, parent)
631 
632         self.results=None
633         self.resultBoxes={}
634 
635         self.header=Header(self,"Results",HEADER_COLOUR)
636         self.header.pack(fill="x")
637 
638         self.homeButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage))
639         self.homeButton.pack(side="right")
640 
641         self.registerButton=ttk.Button(self.header,text="Switch to
    ↪ register layout",style="Header.TButton",command=lambda:parent.
    ↪ show_frame(RegisterPage,self.results))
642         self.registerButton.pack(side="right")
643 
644         #Allows scrolling through the boxes
645         self.boxesFrame=ScrollableFrame(self)
646         self.boxesFrame.canvas.pack(fill="both",expand=True)
647         self.update()
648 
649     def tkraise(self,results,*args,**kwargs):
650         tk.Frame.tkraise(self)
651 
652         while len(self.resultBoxes)>0:
653             ID=next(iter(self.resultBoxes))
654             self.resultBoxes[ID].pack_forget()
655             del self.resultBoxes[ID]
656 
657         self.results=results
658         #Stores the boxes shown in the scrollable frame
659         self.resultBoxes={}
660 
661         #Create the boxes and display project data in them
662         for projectID in results:
663             result=results[projectID]
```

```
664             self.resultBoxes[projectID]=resultBox(self.boxesFrame,self,
    ↪ projectID,result["name"],result["students"],result["size"],result
    ↪ ["genders"],result["skills"])
665             self.resultBoxes[projectID].pack()
666 
667         self.update()
668 
669     #Method override as a work-around for a scrolling bug
670     def update(self):
671         self.master.state('normal')
672         tk.Frame.update(self)
673         self.master.state('zoomed')
674 
675 
676 class RegisterPage(tk.Frame):
677     def __init__(self,parent):
678         #Set up register page layout
679         tk.Frame.__init__(self, parent)
680         self.columnconfigure(0,weight=1)
681         self.columnconfigure(1,weight=1)
682         self.columnconfigure(2,weight=2)
683 
684         self.results=None
685         self.registerBoxes={}
686 
687         self.header=Header(self,"Register",HEADER_COLOUR)
688         self.header.grid(column=0,row=0,columnspan=3,sticky="EW")
689 
690         self.homeButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage))
691         self.homeButton.pack(side="right")
692 
693         self.resultsButton=ttk.Button(self.header,text="Switch to results
    ↪  layout",style="Header.TButton",command=lambda:parent.show_frame(
    ↪ ResultsPage,self.results))
694         self.resultsButton.pack(side="right")
695 
696         #Allows scrolling through the boxes
697         self.boxesFrame=ScrollableFrame(self)
698         self.boxesFrame.canvas.grid(column=2,row=1,sticky="NES")
699         self.update()
700 
701     #Method override as a work-around for a scrolling bug
702     def update(self):
703         self.master.state('normal')
704         tk.Frame.update(self)
705         self.master.state('zoomed')
706 
707     def tkraise(self,results,*args,**kwargs):
708         tk.Frame.tkraise(self)
```

```
709
710         while len(self.registerBoxes)>0:
711             ID=next(iter(self.registerBoxes))
712             self.registerBoxes[ID].grid_forget()
713             del self.registerBoxes[ID]
714
715         self.results=results
716         #Stores the boxes shown in the scrollable frame
717         self.registerBoxes={}
718
719         #Create the boxes and display project data in them
720         for projectID in results:
721             result=results[projectID]
722             self.registerBoxes[projectID]=registerBox(self.boxesFrame,
    ↪ self,projectID,result["name"],result["students"])
723             self.registerBoxes[projectID].grid()
724
725         self.update()
726
727
728 class SettingsPage(tk.Frame):
729     def __init__(self,parent,fitness,iterations,popSize):
730         #Set up swttings page layout
731         tk.Frame.__init__(self,parent)
732         self.fitness=fitness
733         self.iterations=iterations
734         self.popSize=popSize
735
736         self.header=Header(self,"Settings",HEADER_COLOUR)
737         self.header.pack(fill="x")
738
739         self.backButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=self.go_back)
740         self.backButton.pack(side="right")
741
742         self.fitnessVar=tk.DoubleVar(value=self.fitness)
743         self.iterVar=tk.IntVar(value=self.iterations)
744         self.popSizeVar=tk.IntVar(value=self.popSize)
745
746         ttk.Button(self,text="Export database", command=self.export_db).
    ↪ pack()
747
748         #Only allows user to type a decimal number between 0 and 1, no
    ↪ other characters
749         def fitnessCallback(P):
750             if re.match("^0(\\.\\d*)?$",P) or re.match("^1(\\.0*)?$",P)
    ↪ or P == "":
751                 return True
752             return False
753
```

```
754         #Only allows the user to enter positive integers
755         def posIntCallback(P):
756             return str.isdigit(P) or P==""
757 
758         fitnessValidate = (self.register(fitnessCallback), '%P')
759         posIntValidate = (self.register(posIntCallback), '%P')
760 
761         ttk.Label(self, text="Target fitness for genetic algorithm (Must
    ↪ be between 0-1).").pack()
762         ttk.Label(self,text="Higher target fitness means better groups,
    ↪ with 1 being perfect groupings. May not be reached if maximum
    ↪ iterations is reached").pack()
763         ttk.Spinbox(self, textvariable=self.fitnessVar,validate="all",
    ↪ validatecommand=fitnessValidate,from_=0,increment=0.05,to=1).pack
    ↪ ()
764 
765         ttk.Label(self,text="Maximum iterations the genetic algorithm
    ↪ could complete (may be less if target fitness is reached).").pack
    ↪ ()
766         ttk.Spinbox(self,textvariable=self.iterVar,validate="all",
    ↪ validatecommand=posIntValidate,from_=1,increment=5,to=100).pack()
767 
768         ttk.Label(self,text="Size of populations for genetic algorithm.
    ↪ Greater sizes leads to longer runtime, but better results.").pack
    ↪ ()
769         ttk.Spinbox(self,textvariable=self.popSizeVar,validate="all",
    ↪ validatecommand=posIntValidate,from_=1,increment=10,to=200).pack
    ↪ ()
770 
771         ttk.Button(self, text="Save", command=self.save_settings).pack()
772 
773     #Method override that runs once everytime this page is opened
774     def tkraise(self,*args,**kwargs):
775         self.fitnessVar.set(self.fitness)
776         self.iterVar.set(self.iterations)
777         self.popSizeVar.set(self.popSize)
778         tk.Frame.tkraise(self)
779 
780     def get_settings(self):
781         return {"fitness":self.fitness,"iterations":self.iterations,"
    ↪ popSize":self.popSize}
782 
783     def go_back(self): #Only show message when setting has been changed
784         choice=messagebox.askyesno("Warning","Going back will not save
    ↪ your changes. Do you still wish to proceed?",icon="warning")
785         if choice:
786             self.master.show_frame(HomePage)
787 
788     def validate(self):
789         #if re.match("^0(\\.\\d+)?$",) or re.match("1(\\.0*)?$",P):
```

```
790         #Also ensure that popsize >=4
791             return True
792         #return False
793 
794     #Validate and save settings input
795     def save_settings(self):
796         valid=self.validate()
797         if valid:
798             self.fitness=float(self.fitnessVar.get())
799             self.iterations=int(self.iterVar.get())
800             self.popSize=int(self.popSizeVar.get())
801             self.master.show_frame(HomePage)
802             #Update settings.json
803 
804     def export_db(self): #The ui for selecting details before selecting
    ↪ directory
805         self.write_csv()
806 
807     #Save database to a csv file
808     def write_csv(self):
809         filetype=[("CSV file",".csv")]
810         file = filedialog.asksaveasfile(filetypes=filetype,
    ↪ defaultextension=filetype)
811         if file is not None:
812             with sql.connect(DATABASE) as conn:
813                 cur = conn.cursor()
814                 #Fetch names of every table
815                 cur.execute("SELECT name FROM sqlite_schema WHERE type ='
    ↪ table' AND name NOT LIKE 'sqlite_%';")
816                 tables=cur.fetchall()
817                 #Fetch all the data from each table
818                 for table in tables:
819                     data=cur.execute("SELECT * FROM "+str(table[0]))
820 
821                     #Write to the csv
822                     writer = csv.writer(file)
823                     writer.writerow("")
824                     writer.writerow(table)
825                     #Get attribute names
826                     cols=[]
827                     for col in data.description:
828                         cols.append(col[0])
829                     writer.writerow(cols)
830                     writer.writerows(data)
831         else:
832             #Warn the user export was not successful
833             return
834         file.close()
835 
836 #=====WIDGETS=====
```

```
837
838 class Header(tk.Frame):
839     def __init__(self,parent,text,colour):
840         tk.Frame.__init__(self,parent)
841         self.config(bg=colour)
842         self.label = tk.Label(self, text=text, bg=colour,font=LARGE_FONT)
843         self.label.pack(side="left")
844
845 class ScrollableFrame(tk.Frame):
846     def __init__(self,parent):
847         self.canvas=tk.Canvas(parent,background="white")
848         tk.Frame.__init__(self,self.canvas)
849
850         self.scrollbar=tk.Scrollbar(self.canvas,orient="vertical",command
    ↪ =self.canvas.yview)
851
852         self.canvas.configure(yscrollcommand=self.scrollbar.set)
853
854         self.scrollbar.pack(side="right",fill="y")
855         self.scrollbar.update()
856
857         self.canvas.create_window((0,0),window=self,anchor='nw',width=
    ↪ self.winfo_screenwidth()-self.scrollbar.winfo_width())
858         self.canvas.bind("<Configure>",self.scroll)
859         #self.canvas.bind_all("<MouseWheel>", self._on_mousewheel)
860     #def _on_mousewheel(self, event):
861         #self.canvas.yview_scroll(int(-1*(event.delta/120)), "units")
862     def scroll(self,event):
863         self.canvas.configure(scrollregion=self.canvas.bbox("all"),width=
    ↪ self.winfo_width(),height=self.winfo_height())
864
865 #Has checkbox, name, edit button, delete button.
866 class selectionBox(tk.Frame):
867     def __init__(self,parent,page,ID,name):
868         tk.Frame.__init__(self,parent,relief="solid",bd=2)
869
870         self.ID=ID
871         self.name=name
872         self.page=page
873
874         self.selected=tk.BooleanVar(value=False)
875         self.checkbox=ttk.Checkbutton(self,variable=self.selected,onvalue
    ↪ =True,offvalue=False)
876         self.checkbox.pack()
877
878         self.nameLabel=ttk.Label(self,text=name)
879         self.nameLabel.pack()
880
881         self.editButton=ttk.Button(self,text="Edit",command=self.edit)
882         self.editButton.pack()
```

```
883
884         self.deleteButton=ttk.Button(self,text="Delete",command=self.
    ↪ delete)
885         self.deleteButton.pack()
886
887     def check_selection(self):
888         return self.selected.get()
889
890     def edit(self):
891         self.page.editBox(self.ID)
892
893     def delete(self):
894         delChoice=messagebox.askyesno(title="Delete entry", message=f"Are
    ↪  you sure you want to delete {self.name}?")
895         if delChoice:
896             self.selected.set(False)
897             self.page.deleteBox(self.ID)
898
899     def pack(self):
900         tk.Frame.pack(self,fill="x",padx=10,pady=5)
901
902     #-----Abstract methods-----
903     def get_data(self):
904         return
905
906     def edit_data(self):
907         return
908
909
910 class projectBox(selectionBox):
911     def __init__(self,parent,page,ID,name,maxSize,minSize,optional,skills
    ↪ ):
912         selectionBox.__init__(self,parent,page,ID,name)
913         self.maxSize=maxSize
914         self.minSize=minSize
915         self.optional=optional
916         self.skills=skills
917
918         self.maxSizeLabel=ttk.Label(self,text=f"Max group size: {maxSize}
    ↪ ")
919         self.maxSizeLabel.pack()
920
921         self.minSizeLabel=ttk.Label(self,text=f"Min group size: {minSize}
    ↪ ")
922         self.minSizeLabel.pack()
923
924         self.optionalLabel=ttk.Label(self,text=f"Optional: {optional}")
925         self.optionalLabel.pack()
926
```

```
927         self.skillsLabel=ttk.Label(self,text=f"Skills: {", ".join(skills)
    ↪ }")
928         self.skillsLabel.pack()
929 
930     def get_data(self):
931         return {"ID":self.ID,"name":self.name,"maxSize":self.maxSize,"
    ↪ minSize":self.minSize,"optional":self.optional,"skills":self.
    ↪ skills}
932 
933     def edit_data(self,**kwargs):
934         if "name" in kwargs:
935             self.name=kwargs["name"]
936             self.nameLabel.config(text=self.name)
937         if "maxSize" in kwargs:
938             self.maxSize=kwargs["maxSize"]
939             self.maxSizeLabel.config(text=f"Max group size: {self.maxSize
    ↪ }")
940         if "minSize" in kwargs:
941             self.minSize=kwargs["minSize"]
942             self.minSizeLabel.config(text=f"Min group size: {self.minSize
    ↪ }")
943         if "optional" in kwargs:
944             self.optional=kwargs["optional"]
945             self.optionalLabel.config(text=f"Optional: {self.optional}")
946         if "skills" in kwargs:
947             self.skills=kwargs["skills"]
948             self.skillsLabel.config(text=f"Skills: {", ".join(self.skills
    ↪ )}")
949 
950 class studentBox(selectionBox):
951     def __init__(self, parent, page, ID, name, gender, skills):
952         selectionBox.__init__(self,parent,page,ID,name)
953         self.gender=gender
954         self.skills=skills
955 
956         self.genderLabel = ttk.Label(self,text=f"Gender: {gender}")
957         self.genderLabel.pack()
958 
959         self.skillsLabel = ttk.Label(self, text= f"Skills: {", ".join(
    ↪ skills)}")
960         self.skillsLabel.pack()
961 
962     def get_data(self):
963         return {"ID":self.ID,"name":self.name,"gender":self.gender,"
    ↪ skills":self.skills}
964 
965     def edit_data(self,**kwargs):
966         if "name" in kwargs:
967             self.name=kwargs["name"]
968             self.nameLabel.config(text=self.name)
```

```
969         if "gender" in kwargs:
970             self.gender=kwargs["gender"]
971             self.genderLabel.config(text=f"Gender: {self.gender}")
972         if "skills" in kwargs:
973             self.skills=kwargs["skills"]
974             self.skillsLabel.config(text=f"Skills: {", ".join(self.skills
    ↪ )}")
975
976
977 class entryPopupWin(tk.Toplevel):
978     def __init__(self,parent,title):
979         tk.Toplevel.__init__(self,parent)
980         self.title(title)
981
982     def add_string_entry(self,text,variable):
983         ttk.Label(self, text=text).pack()
984         ttk.Entry(self, textvariable=variable).pack()
985
986     def add_int_entry(self,text,variable,minimum=0,maximum=100,increment=
    ↪ 1):
987
988         #Prevents user from typing non numbers into number-only entry
    ↪ boxes
989         def intCallback(P):
990             return str.isdigit(P) or P == ""
991
992         intValidate = (self.register(intCallback), '%P')
993
994         ttk.Label(self, text=text).pack()
995         ttk.Spinbox(self, textvariable=variable,validate="all",
    ↪ validatecommand=intValidate,from_=minimum,increment=increment,to=
    ↪ maximum).pack()
996
997     def add_checkbox(self,text,variable):
998         ttk.Label(self, text=text).pack()
999         ttk.Checkbutton(self, variable=variable, onvalue=True, offvalue=
    ↪ False).pack()
1000
1001    def add_save_cancel(self,window,mode,**kwargs):
1002        ttk.Button(self, text="Save", command=lambda:self.master.
    ↪ save_entry(window,mode,**kwargs)).pack()
1003        ttk.Button(self, text="Cancel", command=self.destroy).pack()
1004
1005 class resultBox(tk.Frame):
1006     count=0
1007     def __init__(self,parent,page,ID,name,students:list,size,genders:dict
    ↪ ,skills:dict):
1008         tk.Frame.__init__(self,parent,relief="solid",bd=2)
1009
1010         resultBox.count+=1
```

```
1011            self.colour=PASTEL_COLOURS[(resultBox.count-1)%6]
1012            self.config(bg=self.colour)
1013
1014
1015            self.page=page
1016            self.ID=ID
1017
1018            ttk.Label(self,text=name,background=self.colour).pack(anchor="w")
1019            ttk.Separator(self).pack(fill="x")
1020
1021            ttk.Label(self,text=f"Students: {", ".join(students)}",background
    ↪ =self.colour).pack(anchor="w")
1022            ttk.Label(self,text=f"Group size: {size}",background=self.colour)
    ↪ .pack(anchor="w")
1023
1024            ttk.Label(self,text="Genders:",background=self.colour).pack(
    ↪ anchor="w")
1025            for gender in genders:
1026                ttk.Label(self,text=f"  > {gender}: {genders[gender]}",
    ↪ background=self.colour).pack(anchor="w")
1027
1028            ttk.Label(self,text="Skills:",background=self.colour).pack(anchor
    ↪ ="w")
1029            for skill in skills:
1030                ttk.Label(self,text=f"   > {skill}: {skills[skill]}",
    ↪ background=self.colour).pack(anchor="w")
1031
1032        def pack(self):
1033            tk.Frame.pack(self,fill="x",padx=10,pady=5)
1034
1035    class registerBox(tk.Frame):
1036        count=0
1037        def __init__(self,parent,page,ID,name,students:list):
1038            tk.Frame.__init__(self,parent,relief="solid",bd=2)
1039
1040            registerBox.count+=1
1041            self.colour=PASTEL_COLOURS[(registerBox.count-1)%6]
1042            self.config(bg=self.colour)
1043
1044            ttk.Label(self,text=name,background=self.colour).pack(anchor="w")
1045            ttk.Separator(self).pack(fill="x")
1046
1047            for student in students:
1048                ttk.Label(self,text=student,background=self.colour).pack(
    ↪ anchor="w")
1049
1050        def grid(self):
1051            tk.Frame.grid(self,column=(registerBox.count-1)%3+1,row=(
    ↪ registerBox.count-1)//3)
1052
```

```
1053
1054
1055
1056
1057
1058
1059 #=====SUBROUTINES=====
1060 def checkForDB(path):
1061     #Database files could have any extension, but they all start with the
     ↪  same first 15 characters.
1062     file=open(path,"rb")
1063     line=file.read(15)
1064     file.close()
1065     #Convert binary to string
1066     line="".join(chr(x) for x in line)
1067     if line == "SQLite format 3":
1068         return True
1069     return False
1070
1071
1072 #=====MAIN=====
1073
1074 app=Window()
1075 app.mainloop()
```

3.2 algoClasses.py

```
1  class Project:
2      def __init__(self,ID,name,skills,maxSize,minSize,optional):
3          self.ID=ID
4          self.name=name
5          self.skills=skills
6          self.maxSize=maxSize
7          self.minSize=minSize
8          self.optional=optional
9
10         self.group=[]
11
12     #Returns count of genders and skills as dicts, and list of students
13     def get_data(self):
14         genders={}
15         skills={}
16         students=[]
17         for student in self.group:
18             if student.gender in genders:
19                 genders[student.gender]+=1
20             else:
```

```
                genders[student.gender]=1

            for skill in student.skills:
                if skill in skills:
                    skills[skill]+=1
                else:
                    skills[skill]=1

            students.append(student.__str__())
        return genders,skills,students

    #Returns fitness value as a float between 0-1
    def getFitness(self):
        fitness=0.0
        stuCount = 0 #Records number of students with at least one
    ↪ relevant skill
        skillsMet=set() #Records the skills the students have that the
    ↪ project demands
        genderCritMet=False #Records whether or not there are multiple
    ↪ genders in the group
        for student in self.group:
            commonSkills=student.skills&self.skills
            #Each student should have one project that is demanded by the
    ↪  project
            if len(commonSkills)>=1:
                stuCount+=1

            #The demands of the project should be met by the students
            skillsMet=skillsMet|commonSkills

            #There should be at least two different genders
            if not genderCritMet and student.gender != self.group[0].
    ↪ gender:
                genderCritMet=True
                fitness+=0.15

        try:
            fitness+=(stuCount/len(self.group))*0.6
        except:
            pass
        finally:
            try:
                fitness+=(len(skillsMet)/len(self.skills))*0.25
            except:
                #If the project has no skill requirements, every group
    ↪ has met every skill requirement so earns the max weight
                fitness+=0.25
        return fitness

    #Adds a student to the project's group
```

```
65      def addStudent(self,students):
66          if type(students)==list:
67              self.group+=students
68
69          elif type(students)==Student:
70              self.group.append(students)
71
72          else:
73              raise TypeError(f"Expected data with type list or Student,
    ↪ got data of type {type(students)}")
74
75      def __str__(self):
76          string = self.name +", "
77          for student in self.group:
78              string += student.__str__() +", "
79          return string
80
81      #Allows for the same project to appear in different individuals
82      def deepcopy(self):
83          return Project(self.ID,self.name,self.skills,self.maxSize,self.
    ↪ minSize,self.optional)
84
85
86  class Student:
87      def __init__(self,ID,name,skills,gender):
88          self.ID=ID
89          self.name=name
90          self.skills=skills
91          self.gender=gender
92      def __str__(self):
93          return self.name
94
95
96  class Individual:
97      def __init__(self):
98          self.projects=dict()
99          self.fitness=0.0
100
101     #Average the fitness values of each project together to return a
    ↪ float between 0-1
102     def calcFitness(self):
103         if self.fitness==-1:
104             return -1
105         total=0.000
106         for project in self.projects.values():
107             total+=project.getFitness()
108         try:
109             self.fitness = round(total/len(self.projects),3)
110         except:
111             self.fitness = 0.000
```

```
112         return self.fitness
113 
114     #Adds a project to the individual's list of projects
115     def addProject(self,projects):
116         if type(projects)==list:
117             for project in projects:
118                 self.projects[project.ID]=project
119 
120         elif type(projects)==Project:
121             self.projects[projects.ID]=projects
122 
123         else:
124             raise TypeError(f"Expected data with type list or Project,
    ↪ got data of type {type(projects)}")
125 
126     def crossover(self,other):
127         return self
128 
129     def mutate(self):
130         return self
131 
132     def __str__(self):
133         string=""
134         for i in self.projects:
135             string+= self.projects[i].__str__()+"\n"
136         return string
137 
138     #Returns the names, size, gender count, and skill count for each
    ↪ project as a dict with the projects' IDs as keys
139     def get_data(self):
140         data=dict()
141         for project in self.projects:
142             project=self.projects[project]
143 
144             genders,skills,students=project.get_data()
145             projectData={"name":project.name,"students":students,"size":
    ↪ len(project.group),"genders":genders,"skills":skills}
146 
147             data[project.ID]=projectData
148 
149         return data
```

3.3 geneticAlgo.py

```
1 from algoClasses import Project,Student,Individual
2 import sqlite3 as sql
3 import random
```

```
4
5  #Fetch student details from database
6  def get_students(database,IDs):
7       students=[]
8
9       with sql.connect(database) as conn:
10           cur = conn.cursor()
11
12           for ID in IDs:
13                #Get each students' skills
14                cur.execute("SELECT Skill FROM StudentSkills WHERE
       ↪ StudentID = ?",(ID,))
15                skills=set()
16                result=cur.fetchall()
17                for skill in result:
18                     skills.add(skill[0])
19
20                #Get each students' name and gender
21                cur.execute("SELECT Name, Gender from Students WHERE
       ↪ StudentID = ?",(ID,))
22                entry=cur.fetchall()[0]
23
24                #Create a Student object for each student with their name,
       ↪  skills, and gender
25                students.append(Student(ID,entry[0],skills,entry[1]))
26
27      return students
28
29
30 #Fetch project details from database
31 def get_projects(database,IDs):
32      projects={"Optional":[],"notOptional":[]}
33
34      with sql.connect(database) as conn:
35           cur = conn.cursor()
36
37           for ID in IDs:
38                #Get each projects' skill requirements
39                cur.execute("SELECT Skill FROM ProjectSkills WHERE
       ↪ ProjectID = ?",(ID,))
40                skills=set()
41                result=cur.fetchall()
42                for skill in result:
43                     skills.add(skill[0])
44
45                #Get each projects' data other than skills
46                cur.execute("SELECT Name, MaxSize, MinSize, Optional from
       ↪ Projects WHERE ProjectID = ?",(ID,))
47                entry=cur.fetchall()[0]
48
```

```
49                #Create a Project object for each project with its name,
    ↪ skills, maxSize, minSize, and whether or not it is optional
50                if bool(entry[3]):
51                     projects["Optional"].append(Project(ID,entry[0],
    ↪ skills,entry[1],entry[2],entry[3]))
52                else:
53                     projects["notOptional"].append(Project(ID,entry[0],
    ↪ skills,entry[1],entry[2],entry[3]))
54
55
56           return projects
57
58 #find optional projects whose minSize is not too large to house the
    ↪ remaining ungrouped students
59 def get_optionals(optionals,extraSlots):
60           optionals.sort(key=lambda x:x.minSize, reverse=True)
61           for i in range(len(optionals)):
62                if optionals[i].minSize>extraSlots:
63                     optionals.pop(i)
64                else:
65                     break
66           return optionals
67
68
69 #Creates an individual for the first population
70 def GenInitialGroups(projects,students):
71           individual=Individual()
72           leftover=[] #Stores projects that have groups, but do not have the
    ↪ maximum number of students assigned yet
73
74           #Fill every required project to minSize with random students
75           required=projects["notOptional"]
76           for i in range(len(required)):
77                required[i]=required[i].deepcopy()
78           for project in required:
79                random.shuffle(students)
80                project.addStudent(students[:project.minSize])
81                individual.addProject(project)
82
83                #Remove grouped students
84                students=students[project.minSize:]
85
86                #Keep projects with room to spare so that they may take on more
    ↪  students later
87                if project.minSize!=project.maxSize:
88                     leftover.append(project.ID)
89
90           #Ensure every student is grouped
91           optionals=[] #Stores projects that do not yet have a group assigned
92           for i in range(len(projects["Optional"])):
```

```
93            optionals.append(projects["Optional"][i].deepcopy())
94
95       while len(students)>0:
96            #Only allow students to be grouped to a project with a minSize
     ↪ that allows the remaining students to be housed
97            optionals=get_optionals(optionals,len(students))
98
99            if len(leftover+optionals)<1:
100                individual.fitness=-1
101                break
102           #Add random students to a random project
103           project=random.choice(leftover+optionals)
104           random.shuffle(students)
105
106           #If the project has no group currently, add minSize number of
     ↪ students
107           if type(project)==Project:
108                optionals.remove(project)
109                project.addStudent(students[:project.minSize])
110                individual.addProject(project)
111
112                students=students[project.minSize:]
113
114                #If project is not full
115                if project.minSize!=project.maxSize:
116                     leftover.append(project.ID)
117
118           #If project already has a group, add one student to it
119           else:
120                individual.projects[project].addStudent(students[0])
121                students.pop(0)
122
123                #Remove from list if now full
124                if len(individual.projects[project].group)>=individual.
     ↪ projects[project].maxSize:
125                     leftover.remove(project)
126       return individual
127
128
129
130 #Returns invalid aspects of an individual i.e. repeated students,
     ↪ projects below minSize, missing students etc.
131 def verifyGroupings(individual):
132       return True
133
134 #Fixes problems flagged by verifyGroupings()
135 def fixGroupings(individual):
136       return individual
137
138
```

```
#Runs the genetic algorithm
def run(database, studentIDs, projectIDs, TARGET_FITNESS, MAX_ITERATIONS,
    ↪  POPULATION_SIZE):
     #List of students
     students=get_students(database,studentIDs)
     #Two lists of projects in dictionary, separated by whether or not
    ↪ the project is optional
     projects=get_projects(database,projectIDs)

     #Ensure there aren't too many or too few students to meet
    ↪ requirements#
     numStudents=len(students)
     minStudents=0
     maxStudents=0

     #Find max and min amount of students required
     for project in projects["notOptional"]:
          maxStudents += project.maxSize
          minStudents += project.minSize
     for project in projects["Optional"]:
          maxStudents += project.maxSize
     #Return error for too few
     if numStudents<minStudents:
          return {"status":-1,"error":f"Error: Only {len(students)} to
    ↪ fill a minimum of {minStudents} places."}
     #Return error for too many
     elif numStudents>maxStudents:
          return {"status":-1,"error":f"Error: {len(students)} is too
    ↪ many to fit in {maxStudents} places."}

     #Generate initial groupings and average fitness
     population = []
     for i in range(POPULATION_SIZE):
          individual = GenInitialGroups(projects, students)
          individual.calcFitness()
          population.append(individual)

     #=====Genetic algorithm======
     for i in range(MAX_ITERATIONS):
          population.sort(key=lambda x:x.fitness,reverse=True)
          #Ends early when an individual has reached the target fitness
          if population[0].fitness>=TARGET_FITNESS:
               return {"status":0,"results":population[0].get_data(),"all
    ↪ ":population}

          #Eliminate individuals in the bottom half in terms of fitness
          population = population[:POPULATION_SIZE//2]
          new_pop = []

          #Create next population
```

```
            crossedSize=int(0.9*POPULATION_SIZE)
            for j in range(crossedSize):
                #Crossover random individuals
                parents=random.sample(population, 2)
                individual=parents[0].crossover(parents[1])

                #Ensure all students appear once
                valid = verifyGroupings(individual)
                if not valid:
                    individual=fixGroupings(individual)

                #Random mutations to prevent convergence towards a local best solution
                individual=individual.mutate()

                new_pop.append(individual)

            #Create the next population with the crossover population and the best 10% of the last population
            population=population[:(POPULATION_SIZE-crossedSize)]
            population+=new_pop
        population.sort(key=lambda x:x.fitness,reverse=True)
        return {"status":0,"results":population[0].get_data(),"all": population}




if __name__=="__main__":
        result=run("Code/database.db",[1,3,4,5,6],[4,5,6],0.0,1,4)
        if result["status"]==0:
            pop=result["all"]
            for ind in pop:
                print(ind.fitness)
                print(ind)
```

3.4 CreateTables.sql

```
-- SQLite
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS Students (
    StudentID INTEGER PRIMARY KEY,
    Name TEXT,
    Gender TEXT
);

```

```
CREATE TABLE IF NOT EXISTS StudentSkills (
    StudentID INTEGER NOT NULL,
    Skill TEXT NOT NULL,
  PRIMARY KEY (StudentID, Skill),
  FOREIGN KEY (StudentID) REFERENCES Students (StudentID)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS Projects (
    ProjectID INTEGER PRIMARY KEY,
    Name TEXT,
    MaxSize INTEGER,
    MinSize INTEGER,
    Optional INTEGER
);

CREATE TABLE IF NOT EXISTS ProjectSkills (
    ProjectID INTEGER NOT NULL,
    Skill TEXT NOT NULL,
  PRIMARY KEY (ProjectID, Skill),
  FOREIGN KEY (ProjectID) REFERENCES Projects (ProjectID)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS Results (
    ResultID INTEGER PRIMARY KEY,
    ProjectID INTEGER NOT NULL,
    LocalTimeGenerated TEXT,
    EpochTimeGenerated INTEGER,
    FOREIGN KEY (ProjectID) REFERENCES Projects (ProjectID)
        ON DELETE CASCADE
        ON UPDATE CASCADE

);

CREATE TABLE IF NOT EXISTS ResultStudentLink (
    ResultID INTEGER NOT NULL,
    StudentID INTEGER NOT NULL,
  PRIMARY KEY (ResultID, StudentID),
    FOREIGN KEY (ResultID) REFERENCES Results (ResultID)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
  FOREIGN KEY (StudentID) REFERENCES Students (StudentID)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

4 Testing

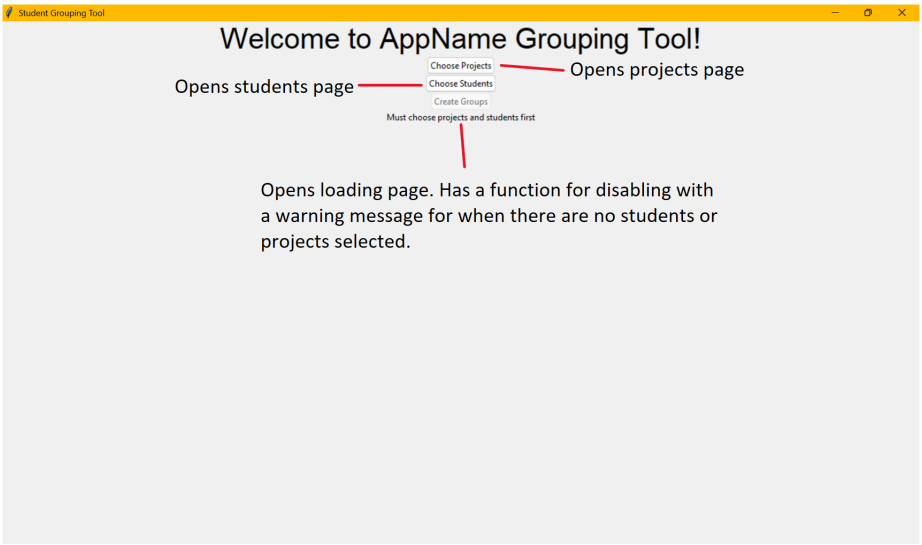


Figure 21: The home page

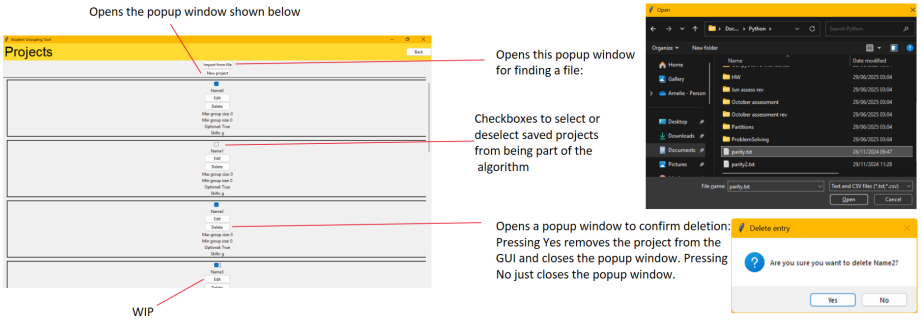


Figure 22: The project page

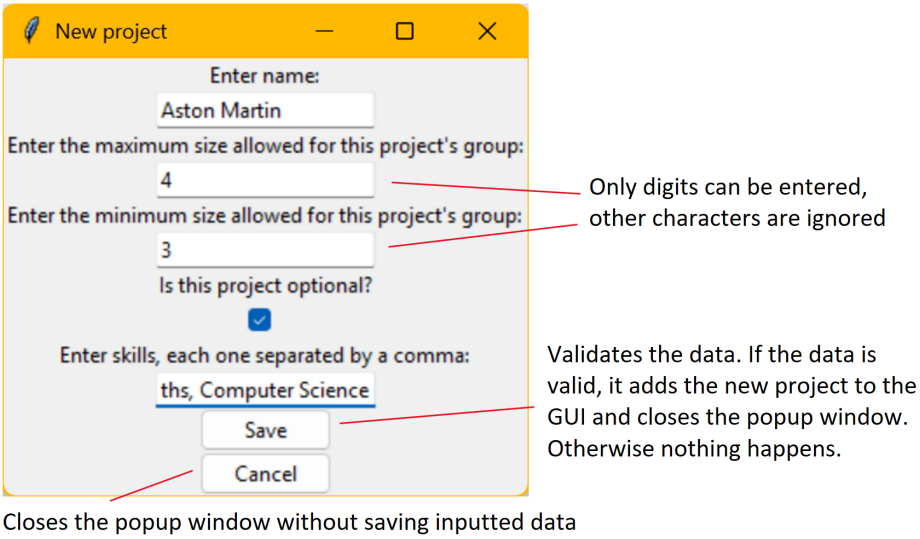


Figure 23: The popup window for entering data about a new project

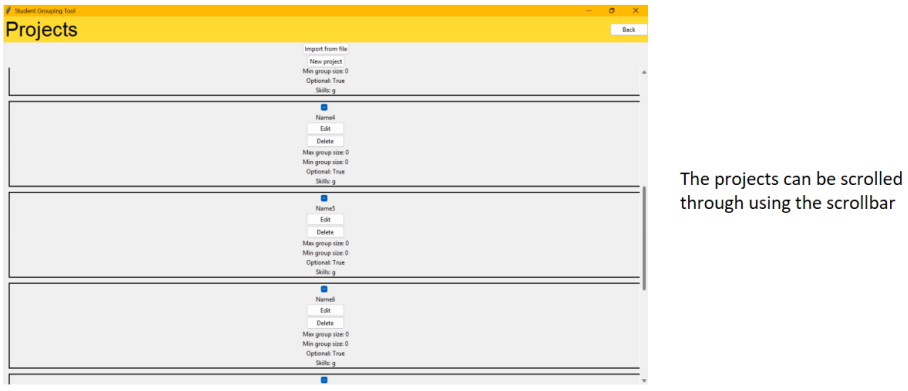


Figure 24: The project page, scrolled down

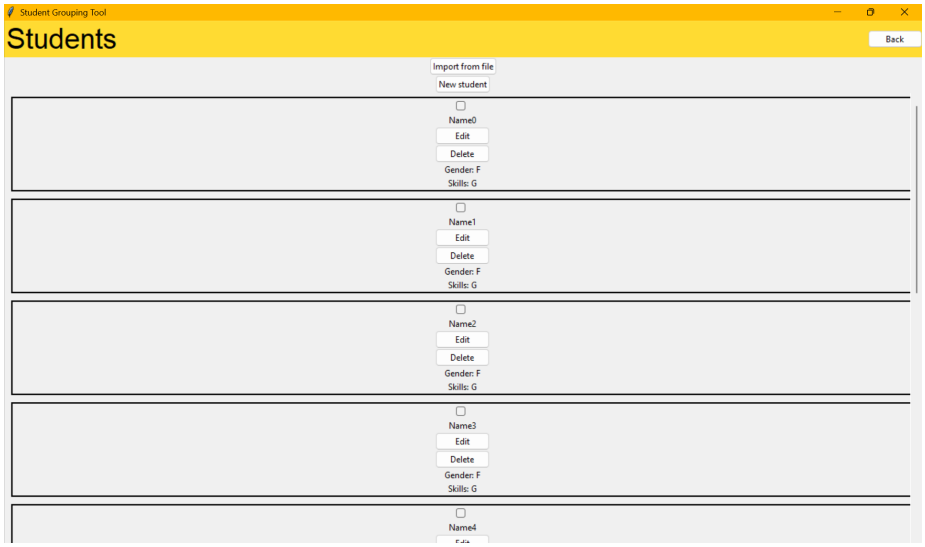


Figure 25: The student page. Functions identically to the projects page

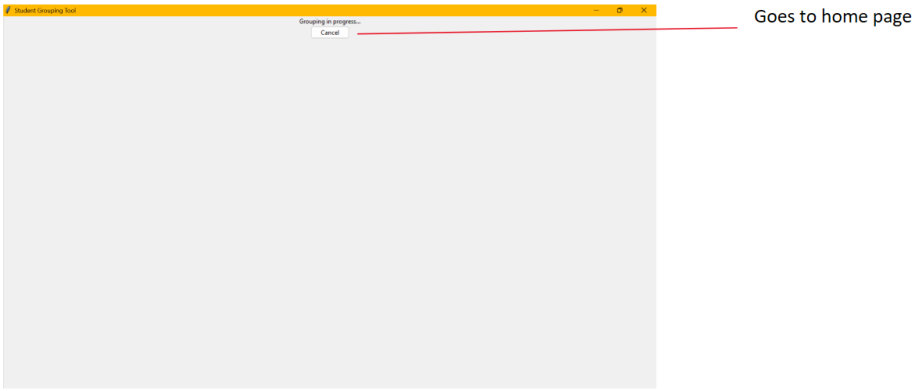


Figure 26: The loading page

4.1 Video

`https://youtu.be/5m5Lk-DWQd8`

4.2 Testing plan

ID	Obj.	Description	Test Data	Expected Results	Time stamp
1	1a,1ei	Check that the student's name must only contain whitelisted characters when entered manually	1.1 (Erroneous): 1.2 (Normal):	1.1: A warning message displays to the user, the pop-up window remains open, and the data does not get saved. 1.2: The pop-up window closes, the data is displayed on the GUI and saved to the database.	
2	1a,1e	Check that all student fields are required	2.1 (Erroneous): 2.2 (Normal):	2.1: A warning message displays to the user, the pop-up window remains open, and the data does not get saved. 2.2: The pop-up window closes, the data is displayed on the GUI and saved to the database.	
3	1a,1e,4a	Check that the entered student data is saved when all fields are completed correctly	(Normal):	An SQL SELECT statement returns the newly added data.	
4	1h,4a	Check that student data can be edited			
5	1h,4a	Check that student data can be deleted			
6	1c	Check that student data is saved from previous uses of the program			

7	1bi,1d	Check that student data can be imported through csv files when all fields are correct			
8	1bi,1ei	Check that student name must only contain whitelisted characters when importing a csv file			
9	1bi,1e	Check that all student fields are required when importing a csv file			
10	1a,1fi	Check that the project's name must only contain whitelisted characters when entered manually			
11	1a,1fii	Check that the project's maximum group size must be greater than or equal to minimum group size when entered manually	11.1 (Erroneous): 11.2 (Boundary): 11.3 (Normal):	11.1: A warning message displays to the user, the pop-up window remains open, and the data does not get saved. 1.2: The pop-up window closes, the data is displayed on the GUI and saved to the database. 1.3: The pop-up window closes, the data is displayed on the GUI and saved to the database.	
12	1a,1fiii	Check that the project's minimum group size must be greater than 0 when entered manually			
13	1a,1f	Check that all project fields are required			

14	1a,1f,4a	Check that the entered project data is saved when all fields are completed correctly			
15	1h,4a	Check that project data can be edited			
16	1h,4a	Check that project data can be deleted			
17	1c	Check that project data is saved from previous uses of the program			
18	1bi,1d	Check that project data can be imported through csv files when all fields are correct			
19	1bi,1fi	Check that project name must only contain whitelisted characters when importing a csv file			
20	1bi,1fii	Check that the project's maximum group size must be greater than minimum group size when importing a csv file			
21	1bi,1fiii	Check that the project's minimum group size must be greater than 0 when importing a csv file			
22	1bi,1f	Check that all project fields are required when importing a csv file			
23	1gi	Check that setting's target fitness must be between 0 and 1			

24	1gii	Check that setting's max iterations must be a positive integer, not including 0			
25	1giii	Check that setting's size of populations must be a positive integer, not including 0			
26	4bi	Check that database can be exported as a csv file			
27	4c	Check that settings data is stored as a JSON			
28	1i	Check that at least one student, one project, target fitness, max iterations, and size of populations must all be entered before the algorithm is run			

5 Evaluation

5.1 Objective evaluation

The program successfully takes in student and project data, validates it, runs a grouping algorithm, and outputs the results onto a GUI.

[Testing showed that this objective was achieved/not achieved for multiple objectives]

5.2 Third-party feedback

1.

5.3 Feedback evaluation

5.4 Improvements